基于目标优化与关联因素分析的农作物种植安排决策模型

摘要

种植产业作为乡村经济的核心产业，是发展乡村振兴的重要一环。因此，因地制宜地有效规划种植策略是实现有机种植产业可持续发展所需面对的问题。本文针对华北地区某乡村的历史数据与作物和地形等信息，在满足题目所给条件前提下，以收益最优为目标，结合启发式算法与相关性分析等方法，在不同因素影响下找到理想的种植策略。

针对问题一，首先进行数据处理，本文通过查阅官方资料，**计算得到 2023 年除蔬菜类与食用菌类作物滞销率 1.08%，其余滞销率为 0%**，由此计算得到作物预期销售量。下一步，在针对目标最大化的优化算法设计方面，本文结合题设条件与数据特点，选择使用遗传算法以模拟生物进化的过程，实现方案的优化。通过生成满足约束条件的初始化个体，进行数据的交叉与变异操作，利用轮盘赌法选择基因的遗传与舍弃，在迭代中逼近最优解方案。

针对问题二，题目中种植成本、销售价格、预期销售量等因素的波动决定其为一**不确定参数优化问题**。使用随机生成正态分布数集方式确定参数在一定程度上会导致种植策略的波动性增大，同时方案的适应性也将减弱。因此，本文选择使用以**鲁棒优化**为主体，结合最优化环境条件作为对比参考的模型设计，将参数设置为最不利于目标最大化的大小，将不确定优化转化为一般优化问题。同时，所得到极端情况下的最优解使得模型具有良好的鲁棒性，避免了农业生产过程的未知风险因素对农户个体与乡村集体造成损失，也为未来进一步研究市场不确定因素提供可行方案。此外，除了题目所提到的不确定因素，**本文还将灾害因素等纳入考虑，并对其进行数据正态分布检验**，增强了模型应用的广泛性。

针对问题三，根据题目，预期销售量与销售成本、种植成本之间存在一定的相关性。首先对数据分布情况使用 **JB(Jarque-Bera test) 正态分布检验**，结果为非正态分布。因此采用 **Spearman 检验**其相关性。此外，在分析其具体数值关系时，由于仅有 2023 年一期数据，且农作物品类与附加值等多因素干扰，分别对预期销售量与销售成本、种植成本进行半对数拟合，然而拟合优度并不高。对此本文采用**系统聚类与 K-means++ 结合的方法**将农作物进行聚类，再对两类分别拟合得到关系曲线，拟合优度大幅度上升，例如，经过聚类分析后，第二类相关指标的拟合效果相较之前增长了 32.9%。最后将拟合函数应用于优化算法中，结果表明收益值有所提高。对于农作物的替代性与互补性的研究，本文通过对农作物大类的替代性与互补性矩阵，**刻画了通过往期种植策略约束未来种植策略的约束条件**，并模拟数据编程，为农作物替代性与互补性对于种植策略的优化提供了思路。

关键字： K-means++ 聚类分析　遗传算法　鲁棒优化　半对数拟合

一、 问题重述

1.1 问题背景

在当今农业持续进步的时代背景下，如何高效且可持续地管理有限的耕地资源，通过优化种植布局来提升农业经济效能，已成为乡村经济可持续增长所必须深入探讨的关键问题。特别是在诸如华北山区等特定地域，鉴于其独特的气候环境、多样化的土地资源构成以及作物生长的特定规律，精心策划农作物的种植策略变得尤为关键。这种规划不仅是对自然条件的适应，更是推动农业可持续发展、增强农村经济活力的必要途径。

1.2 问题内容

本研究以华北山区某乡村的农作物种植策略为研究对象，基于历史农作物的种植情况、亩产量、种植成本和销售单价等相关历史统计数据，农作物生长特性及现有耕地相关指标等信息，建立数学模型，以解决以下关键问题：

（1）**问题一**：在农作物预期销售量、种植成本、亩产量和销售价格均保持稳定相对2023年且每季农作物需当季销售完毕的前提下，求解2024～2030年农作物的最优的种植策略。相关数据位于附件1与附件2中，对超出预期销售量的部分做以下处理：

- 情况一：超出预期销售量的部分将滞销，造成浪费。
- 情况二：超出部分按2023年销售价格的50%降价处理。

分别计算并输出两种情况下的最优种植方案至result1_1.xlsx和result1_2.xlsx。

（2）**问题二**：考虑农作物未来预期销售量、亩产量、种植成本和销售价格的不确定性，其中小麦和玉米预期销售量年平均增长率为5%-10%，其他作物预期销售量相对于2023年有±5%变化；亩产量受气候影响每年有±10%变化；种植成本每年增长约5%；粮食类作物价格稳定，蔬菜类作物价格每年增长约5%，食用菌价格稳中有降（特别是羊肚菌每年降5%）。综合考虑上述不确定性及风险，制定2024-2030年农作物的最优种植方案，并输出至result2.xlsx。

（3）**问题三**：在问题2的基础上，进一步考虑农作物之间的可替代性和互补性，以及预期销售量、销售价格与种植成本之间的相关性。通过模拟数据，综合考虑这些复杂因素，制定2024-2030年农作物的最优种植策略，并与问题2的结果进行比较分析。

二、 问题分析

2.1 问题一的分析

问题一要求分析附件 1 和附件 2 提供的数据，针对两种情况分别得出从 2024 年到 2030 年该村农作物的最优种植方案，即得到使得利润最大的种植方案。针对两种情况可以得到两个利润的目标函数，加上题干中每种作物在同一地块（含大棚）都不能连续重茬种植，每个地块（含大棚）的所有土地三年内至少种植一次豆类作物等约束条件。故可以选择以遗传算法进行优化学习，在约束条件下随机生成满足要求的初始种群后模拟生物进化的染色体交叉与基因突变等操作对种群进行选择与遗传，并代入附件中的相关数据，从而得出最优种植方案。

同时，由于附件中缺少 2023 年各作物每季的预期销售量，需要提前查询相关数据文献，以便模型求解。

2.2 问题二的分析

第二问在第一问的基础上增加了对于不确定因素与风险因素等的考虑，预期销售量、农作物亩产量、销售价格和种植成本等指标转变为波动变量，在一定区间内变化。对于不确定性优化问题，本文可以选择使用鲁棒优化的方式条件转化为确定性问题，减少环境扰动对模型优度的影响，并且增强在恶劣条件下的策略适应性，同时条件数据变化范围，题目中已经告知。但仍存在其他潜在风险需要纳入考虑，譬如：自然风险，运输风险等等。这些潜在风险则需要查询相关数据，进一步对目标函数进行约束。

2.3 问题三的分析

问题三已经指出预期销售量、销售价格与种植成本之间存在一定相关性。那么意味着可以通过相关性分析与数据拟合等方式定性定量地认识三者间的相关性。有了定量关系后，再在问题二的基础上综合考虑。假定种植成本选取问题二条件下农作物种植成本，再根据拟合获得的定量关系，问题二中动态浮动的预期销售量和销售价格将趋于稳定，即两者的数值此时是由种植成本所决定的由此生成所需测试模拟数据。再基于问题二的算法进行简单改造计算对应最优种植策略。农作物相关数据与第二问发生了改变，最优策略也将发生改变。

因为拟合函数本质是考虑进去了各种农作物的可替代性和互补性，预计此时最优种植策略相比于第二问的种植策略，逐年相比总利润是增加的。具体解后具体分析。

三、 模型的假设

- 通过分析 2023 年的种植策略特点，假设仅能单季种植的 26 个地块每年仅种植单一作物，其余 28 个地块每个季度仅种植两种作物，且种植面积平均分配，以满足每种作物在单个地块种植面积不宜太小，同时方便管理的需求，并简化模型。
- 假设题干所述华北地区该地一切贸易仅考虑国内市场，因为该地处于华北山区农村地带，销量小，无法有有效的渠道进入国际贸易市场。
- 忽略 2023 年以前各作物, 尤其是豆类的种植情况，以简化模型。
- 假设华北地区潜在种植风险只考虑自然风险，并且可以量化。
- 假设各个地块在种植策略规划年间均可正常进行种植，不过度受自然因素与人文因素的影响。
- 假设该地区的种植技术水平保持稳定，在种植策略规划年间不发生明显变化。

四、 符号说明

对文中所用到的符号加以阐述和说明，如表 1 所示。

表 **1**　符号说明

符号	说明
r	变化率
t	年份
s	各种类型地的面积
k	地的种类编号
x	作物的种类编号
W	利润
Sale	预期销售量矩阵
Price	销售价格矩阵
Yield	亩产量矩阵
Field	土地规划矩阵
Cost	种植成本矩阵

五、 模型的建立与求解

5.1 问题一的模型建立与求解

5.1.1 数据处理

1. 获取作物预期销售量数据

由于附件中缺少 2023 年各作物每季的预期销售量，所以首先进行农产品预期销售量的测算。预期销售量，即人们对作物的需求量。因为地处华北山区村庄，销量较小，没有较好的渠道进入国际贸易，所以只考虑国内市场的需求。

粮食自给率是一项评估国家粮食自给程度的指标，用总产量/（总产量 + 进口量-出口量）定义。通过判断自给率是否大于 1，可以分为两种情况：

- 当自给率 >100% 时, 净出口量即为国内市场的农作物滞销量，滞销率 = 滞销量/总产量。
- 当自给率 <100% 时, 总产量 = 预期销售量，滞销率为 0。

总产量抛去滞销量，那么剩下的就是预期销售量，即预期销售量 = 产量 ×(1-滞销率)。

从《中国统计年鉴 (2023)》得到薯类、高粱等粮食类以及菜和食用菌相关数据 [1]。最终得到粮食类农作物自给率为 86.26%, 滞销率为 0，菜及食用菌类农作物自给率为 101.09%，滞销率为 1.08%。从而得到作物的预期销售量。

2. 初始化各种数据矩阵及规划矩阵表示形式

通过附件中信息的获取，最终的得到了 Sale, Price, Yield, Cost 四种矩阵的初始数据。四种矩阵形式上都是 41×6，分别对应 41 种作物在六类地形 (包括大棚) 中对应的统计量，矩阵每一行都代表一个作物。相关统计数据置于支撑材料中。六类地形在矩阵中的排列顺序依次为平旱地、梯田、山坡地、水浇地、普通大棚、智慧大棚。

对于最终模型结果的规划矩阵，使用 Field 矩阵表示，如下：

$$\text{Field} = \begin{bmatrix} x_{1,1} & 0 & 0 & 0 & s_1 & k_1 \\ x_{2,1} & 0 & 0 & 0 & s_2 & k_2 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_{m,1} & x_{m,2} & x_{m,3} & x_{m,4} & s_m & k_m \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_{54,1} & x_{54,2} & x_{54,3} & x_{54,4} & s_54 & k_54 \end{bmatrix}$$

Field 为 54×6 大小的矩阵，其中每一行代表一个地块，每一行的六个元素为相应地块策划种植信息。根据假设，前 26 个地块均种植单一作物，由每一行第一个数值表示作物编号；后 28 个地块可至多种植两个种类的作物且种植面积平均，每一行前两个数值

表示第一季度种植的作物的编号，第三、四个表示第二季度种植的编号。s 和 k 分别表示地块对应亩数大小和地块类型编号，平旱地、梯田、山坡地、水浇地、普通大棚、智慧大棚从 1-6 依次标号。

5.1.2 模型建立

1. 约束条件

(1) 平旱地、梯田和山坡地每年都只能种植一季作物。

(2) 水浇地每年可以种植一季也可以种植两季作物。

(3) 大棚能够在一定程度上起保温作用，每年都可以种植两季作物。

(4) 智慧大棚主要是在冬季利用太阳能自动调节棚内温度，保证作物的正常生长。

(5) 各类地块可种植的作物种类存在差异，其中水稻的种植还会影响到水浇地可进行种植的时间。

(6) 每种作物在同一地块 (含大棚) 都不能连续重茬种植，即 $\text{Field}(t) \neq \text{Field}(t-1)$。

(7) 因含有豆类作物根菌的土壤有利于其他作物生长，从 2023 年开始要求每个地块 (含大棚) 的所有土地三年内至少种植一次豆类作物，即设豆类集合为 L，则：

$$\begin{cases} Field(t) \neq Field(t-1) \\ \sum_{k=t-2}^{t} 1, Fieldi, j \in L \end{cases}$$

对于 (1)-(4) 隐含的约束条件在规划矩阵 Field 的结构设置中已经囊括，无需额外进行显式限制条件的增加。同时，题干中对于每种地形存在据具体的种植限制，主要有 (5) 约束，模型将通过条件判断与数据监测加以限制。此外，根据作物的生长规律，对于相邻年份来说作物种植会有所影响, 所以产生了 (6)(7) 两条约束。

2. 目标函数

由于题干要求分析两种特定情况下的最优方案，即保证它们的利润最大。故分别列出两种情况的目标函数：

$$max \quad W_1 = \sum_{i}^{54} \sum_{j}^{4} min(\frac{\text{Yield}_{m,n} \times \text{Field}_{i,5}}{2}, \text{Sale}_{m,n}) \times \text{Price}_{m,n} - \frac{\text{Cost}_{m,n} \times \text{Field}_{i,5}}{2}$$

$$\begin{aligned} max \quad W_2 = & \sum_{i}^{54} \sum_{j}^{4} min(\frac{\text{Yield}_{m,n} \times \text{Field}_{i,5}}{2}, \text{Sale}_{m,n}) \times \text{Price}_{m,n} - \frac{\text{Cost}_{m,n} \times \text{Field}_{i,5}}{2} \\ & + \left| \frac{\text{Yield}_{m,n} \times \text{Field}_{i,5}}{2} - \text{Sale}_{m,n} \right| \times \frac{\text{Price}_{m,n}}{2} \end{aligned}$$

其中，$m = \text{Field}_{i,j}, n = \text{Field}_{i,6}$。

3. 模型建立

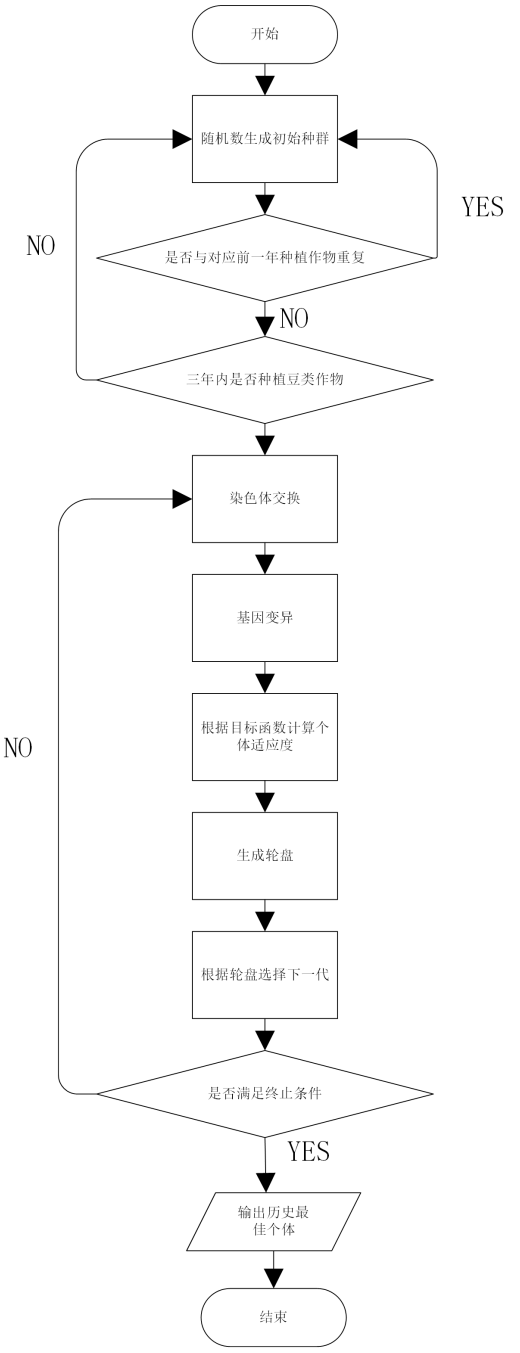


图 **1**　遗传算法模型流程图

结合题目数据特点与结构，本文将该问题视为一道在复杂约束条件限制下，进行目标函数最大化的优化问题，选择采用遗传算法进行求解，通过模拟生物进化过程进行交叉变异等操作以得到最优解。, 具体思路如图 1 所示。

创建初始种群个体时，通过设置限制：

(1)Field$(t) \neq$ Field$(t-1)$;

(2)Field(t), Field$(t-1)$, Field$(t-2)$ 中存在对应可种植的豆类食物。

以实现同一种作物不可连续在同一地块连续种植的约束且同一地块三年内必至少种植一次豆类作物。

此外，在随机数生成初始种群部分，本文通过对不同地块情况的分类讨论，添加相应的限制条件，实现各地块在相应季度种植正确的作物。

下一步，模拟染色体交换的生物过程，以各种植策略矩阵为单位，设置 70% 的交换发生概率，对生成随机数所指定的地块间进行种植策略的交换，同时进行方案检测，保证各种植策略矩阵在进行交换后仍满足约束条件。

接下来，模拟生物的基因变异过程，设置 10% 的变异发生概率，以地块为对象进行重新随机赋值，同时如上一步进行方案检测。

最后，计算各种植策略矩阵对应方案下的收益，即适应度。再通过轮盘赌法，保存最优个体，淘汰较差的个体，并随机生成新个体进入种群。上述步骤迭代多次至最优解稳定且理想，根据实际测试，迭代次数选择设置为 200。

5.1.3 模型求解

运用 python 编写模型，将数据带入模型进行求解，分别得到两种情况下 2024-2030 年的最优农作物种植方案。最终方案分别已经存入 result1_1.xlsx 和 result1_2.xlsx，置于支撑材料中。

5.1.4 模型评价

图 2 选取展示了按照目标函数 1 计算得到的 2024 年的历史最优个体的变化图。

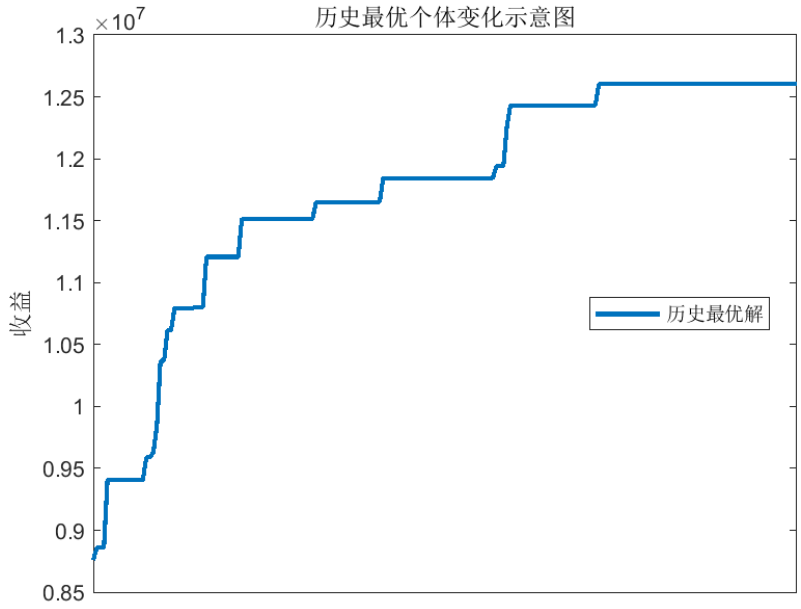


图 **2**

通过观察可以发现，经过逐代的遗传、交叉、变异等操作，历史最优个体对应的适应度，即收益逐渐上升并最终趋于稳定。通过模拟生物进化的方式方案逐渐趋于全局最优解，并且基因变异等操作有效地避免了最终方案进入局部最优解的循环当中，大大提升了优化模型的灵活性与适应性。

5.2 问题二的模型建立与求解

5.2.1 数据处理与分析

对于不确定因素下的优化问题，本文采用鲁棒优化与最优环境条件优化相结合的方法，分别讨论在参数设置最有利于目标函数增长和最不于目标函数最大化目标实现的两种情况下，种植策略与最终受益的差异，为作物种植策略的设定提供更加全面的参考。此处重点介绍鲁棒优化的方法：在最坏情况下目标函数的函数值达到最优，使得该解在

表 **2**　参数取值表

优化类型	最优环境优化	鲁棒优化
小麦和玉米预期销售量变化率	10%	5%
其他作物相对 2023 预期销售量变化率	5.00%	-5.00%
亩产量变化率	10%	-10%
种植成本变化率	6.50%	3.50%
粮食类销售价格变化率	0%	0%
蔬菜类销售价格变化率	6.50%	3.50%
食用菌类销售价格变化率	-1%	-5%
羊肚菌销售价格变化率	-5%	-5%
2025 年收益	14859462	5804243

可能出现的所有情况下，约束条件均能被满足，同时也增加了方案的可行性，减少了参数扰动对于种植策略的影响。

在题设条件下，分别选择最有利和最不利于目标函数最大化的参数，并选取 2025 年的销量在两种条件上产生的最优种植策略所对应的收益制成表 2 所示。

此外，除了预期销售量、农作物亩产量、种植成本以及销售价格的不确定性，还有许多的潜在种植风险值得去考虑。近年来，受不断恶劣的极端气候影响，我国农作物受灾、成灾、绝收面积不断变化。

农作物受灾面积指：农作物比正常年份减产 10％以上的播种面积。（此处记作 10%）

农作物成灾面积指：农作物比正常年份减产 30％及以上的播种面积。（此处记作 30%）

农作物绝收面积指：农作物比正常年份减产 70％及以上的播种面积。（此处记作 70%）

据国家统计局编写的《中国统计年鉴 2023》[1] 数据表示，近十年来的自然灾害所导致的受灾、成灾、绝收面积高居不下，具体如表 3 所示：

灾害损失率的计算方法如下：灾害损失率 =(耕作面积-10%× 受灾面积-30%× 成灾面积-70%× 绝收面积)/耕作面积 ×100%

在此基础上，为了反映自然情况下的灾害损失率大小，预想构造满足某正态分布的随机量，以得到不同年份下的灾害损失率。然后针对灾害损失率随机量进行夏皮洛-威尔克检验，判断在 95% 的置信水平下，是否能拒绝原假设 H_0：该随机量服从正态分布。

表 3　近十年自然灾害导致受灾、成灾、绝收面积 (单位：千公顷)

年份	受灾面积	成灾面积	绝收面积	耕作面积	灾害损失率
2023	10539	4797	980	171624.47	0.981
2022	12072	4373	1351.8	169990.92	0.98
2021	11739	4682	1632.8	168695.13	0.978
2020	19958	7993	2706	167487.14	0.962
2019	19257	7913	2802	165930.66	0.962
2018	20814	10569	2585	165902.38	0.957
2017	18480	9200	1826.7	166331.91	0.965
2016	26220	13670	2902.2	166939.04	0.948
2015	21770	12380	2232.7	166829.28	0.955
2014	24890	12680	3090.3	165183.32	0.949
平均值	18573.9	8825.7	2210.95	167491.4	0.963749

正态性检验结果位于附录中，得到无法拒绝原假设的结果，所以灾害损失率随机量服从期望为 0.9637，方差为 0.00151 的正态分布，即 $X \sim N(0.9637, 0.000151)$。最后以此为依据生成不同年份下的灾害损失率，与产量相乘参与计算。

5.2.2 模型建立

1. 目标函数

在算法设计上，基本沿袭第一问的算法。对于超出预期销售量的部分同第一问的第一种情况进处理，即超过部分滞销，造成浪费。同时对将目标函数修改为如下：

$$max \quad W_1 = \sum_{i}^{54}\sum_{j}^{4} min(\frac{\text{Yield}_{m,n} \times \text{Field}_{i,5}}{2} \times 0.9^{t-2023}, \text{Sale}_{m,n} \times 0.95) \times \text{Price}_{m,n} \times 1.05^{2023-t} - \frac{\text{Cost}_{m,n} \times \text{Field}_{i,5} \times 1.05^{t-2023}}{2}$$

其中，$m = \text{Field}_{i,j}, n = \text{Field}_{i,6}$。$t$ 表示年份。此处参数设置仅对大部分作物与地块有效，部分特殊对象在算法中单独进行处理。

2. 约束条件

$$\text{Sale}_{i,j}(t) = \text{Sale}_{i,j}(t-1) \times (1 + r_{i,j}), r_{i,j} \in [-0.05, 0.05], i \neq 6, 7$$

$$\text{Sale}_{i,j}(t) = \text{Sale}_{i,j}(t-1) \times \left(1 + r_{i,j}^{t-2023}\right), r_{i,j} \in [0.05, 0.1] i = 6, 7$$

$$\text{Yield}_{i,j}(t) = \text{Yield}_{i,j}(t-1) \times \left(1 + r_{i,j}^{t-2023}\right), r_{i,j} \in [-0.1, 0.1]$$

$$\text{Cost}_{i,j}(t) = \text{Cost}_{i,j}(t-1) \times \left(1 + r_{i,j}^{t-2023}\right), r_{i,j} \in [0.045, 0.055]$$

$$\text{Price}_{i,j}(t) = \text{Price}_{i,j}(t-1) \times \left(1 + r_{i,j}^{t-2023}\right), r_{i,j} = 0.05, i = 17, 18 \ldots 37$$

$$\text{Price}_{i,j}(t) = \text{Price}_{i,j}(t-1) \times \left(1 + r_{i,j}^{t-2023}\right), r_{i,j} \in [-0.05, -0.01], i = 38, 39, 40$$

$$\text{Price}_{i,j}(t) = \text{Price}_{i,j}(t-1) \times \left(1 + r_{i,j}^{t-2023}\right), r_i, j = -0.05, i = 41$$

$$\text{Price}_{i,j}(t) = \text{Price}_{i,j}(t-1), i = 1, 2, \ldots 16$$

5.2.3 模型求解

运用问题一编写的 python 模型，修正目标函数，将数据带入模型进行求解，得到此刻情况下 2024-2030 年的最优农作物种植方案。最终方案已经存入 result2.xlsx，置于支撑材料中。

5.3 问题三的模型建立与求解

依据题意，为了进一步考虑作物之间的相互关系，既可以从农作物间可替代性与互补性方面去改善最优种植策略，也可以从农作物预期销售量、销售价格与销售成本间相关性去改善模型。对于前者，本研究建立了一种基于 Lingo 的传统规划算法的数学模型，并选取部分数据进行模型求解，仅提供一种思路。本题主要着重于通过后者：农作物相关统计量之间的相关性进行具体建模分析。

5.3.1 基于作物可替代性与互补性的种植方案分析

1. 量化作物间的可替代性与互补性

为了简化模型，该模型仅针对作物类型：粮食，粮食豆类，蔬菜，蔬菜豆类，食用菌进行可替代性与互补性间的量化。五类作物依次记作：A_1, A_2, A_3, A_4, A_5，通过专家咨询等多方面考究制定了表 4 。

表 **4**　作物替代性与互补性量化表

A_1	A_2	A_3	A_4	A_5	A_6
A_2	0	-0.3	0.5	0.3	0.6
A_3	-0.3	0	0.4	0.2	0.4
A_4	0.5	0.4	0	-0.1	-0.2
A_5	0.3	0.2	-0.1	0	0.4
A_6	0.6	0.4	-0.2	0.4	0

正值表示两类作物间具有互补性，负值表示两类作物间具有替代性。任何两类作物间一定是由一定的替代性或者互补性，只不过有关系强弱之分。数值绝对值越大，作物之间相应的性质就越强。比如 A_1(粮食) 和 A_2(粮食豆类) 对应的量化值为-0.3，意味着他们有一定的互补性，那么当粮食的销量增加时，粮食豆类的销量就会在一定程度上下降；A_4(蔬菜豆类) 和 A_5(食用菌) 的对应值为 0.4，那么当二者某一类作物销量上升时，另一类作物销量也将上升。

此时，量化值就不单只是替代互补关系的象征，它也可以作为某类作物销量改变时，对其他作物销量改变趋势与大小的象征。

2. 连续年份各类作物间销量的解析表示

考虑到供给关系间波动具有延迟性，所以对于某类作物销量改变时，其他作物销量一定会之后有所浮动。考虑到题干背景处于华北地区某乡村，地域范围小，销量是的来源主要来自村民本年度的种植。若想改变销量以获利，故有理由假定，波动的延迟性在 1 年左右。

倘若不考虑其他作物影响，在种植面积相同的情况下，只考虑作物 A_1 和 A_2 的销量，那么可以得到关系 $A_1(t+1) = A_1(t) + k\omega A_2(t)$。其中 ω 即表中 A_1 和 A_2 关系的量化值。t 表示年份，即次年的销量可以用前一年的销量进行表示。k 为限制系数，因为地域环境，政策等多种原因，不同地区作物间替代互补关系对销量的影响大小是有区别的，故采用系数 k 进行限制。

若考虑多种作物销量对于来年对于 A_1 销量的影响，将上式拓展求和即可。如下所示：

$$A_1(t+1)S_1(t+1) = A_1(t)S_1(t) + k\sum_{i=2}^{5}(\omega_{1,i}A_i(t)S_i(t))$$

其中，$S_i(t)$ 表示 A_i 作物在第 t 年的种植面积。

3. 模型建立与求解

保证土地总面积一定，预期销量和调整产量都不低于总产量。使得总利润最大即可。

为了简化模型，假设每类植物初始土地种植面积相等为 200 亩，五类作物分别从附件中选取小麦、黄豆、菠菜、豇豆、白灵菇，以他们 2023 年的的成本、售价、亩产量在 Lingo 规划得到 2024 年这五类作物作物的最优培养方案。

Lingo 最终求解培养方案已置于附录中。

5.3.2 基于作物预期销售量、销售价格与销售成本相关性的种植方案分析

1. 数据相关性分析

表 **5**　描述统计

	最小值	最大值	均值	方差	偏度	峰度
种植成本/(元/亩)	350	10000	1890.65	3636363.511	2.605	9.143
销售单价/(元/斤)	1.50	100.00	9.0742	205.306	5.137	28.946
预期销售量/斤	810	150000	20165.89	780961176.659	2.553	7.966

为了得到预期销售量、销售价格和种植成本之间的相关性，研究选取附件提供的 2023 年各类农作物的相关数据进行数据分析，有表 5 所示结果。

很明显农作物的三类统计量都出现方差、偏度、峰度较大的现象。尤其是销售单价一栏，反映出农作物单价分布不对称，高价农作物占多，拉动整体价格水平；并且整体分布不均匀，稳定性很差。这跟当地作物多样性，作物属性关系联系很大。

在进行统计量相关性检验的过程中，可选择 Pearson 相关系数与 spearman。Pearson 相关系数是衡量线性相关的一种良好指标，但其假设检验成立条件需要数据通常是成对的来自于正态分布的总体，才能使用 t 检验的方法来进行相关假设性检验。所以模型需将先一步对种植成本、销售单价和预期销售量这三个统计量进行正态分布检验。

由于样本数较大，所以选择采取 JB 检验 (Jarque-Bera test) 去判断数据是否属于正态分布。对于某随机变量 x_i，已知其偏度为 S，峰度为 K，可构造 JB 统计量：$JB = \frac{n}{6}[S^2 + \frac{(K-3)^2}{4}]$, 可以证明，如果 X_i 服从于正态分布，那么在大样本情况下 $JB \sim \chi^2(2)$（自由度为 2 的卡方分布）。进而进行假设性检验，原假设 H_0：随机变量 X_i 服从于正态分布。通过该变量的偏度和峰度，得到检验值 JB^*, 并计算出对应 p 值与 0.05 比较，若小于 0.05 可拒绝原假设，反之则不能。通过 matlab 计算得到它们的 p 值都是 10^{-3} 量级的，远小于 0.05。故拒绝原假设，这三类统计量都不遵循正态分布。同时，做出了 QQ 图加以佐证，它们并不遵循正态分布, 且两两之间并非一定为线性关系, 由此无法选择 Pearson 相关系数。图 3 仅列出销售单价的 qq 图及其去趋势图。

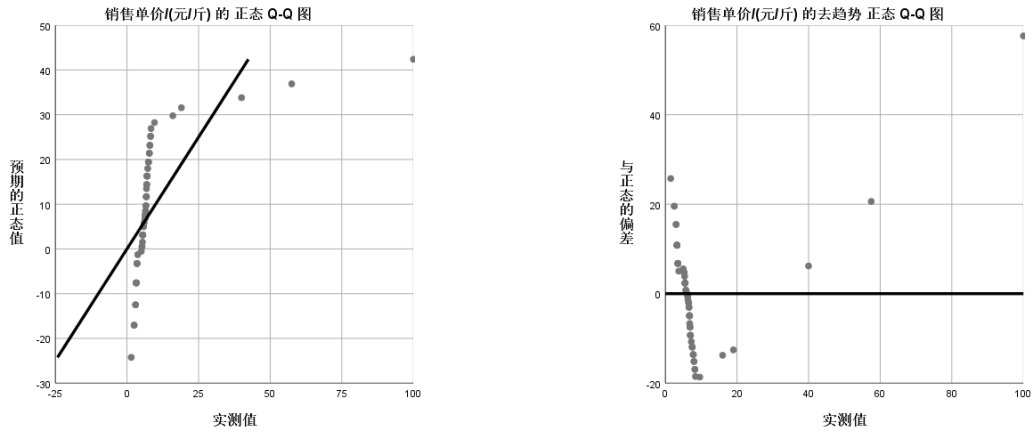


图 **3**　销售单价 **qq** 图及其去趋势图

所以最终选择了斯皮尔曼相关系数来判断这三类统计量两两之间的相关性系。如图 4 所示。

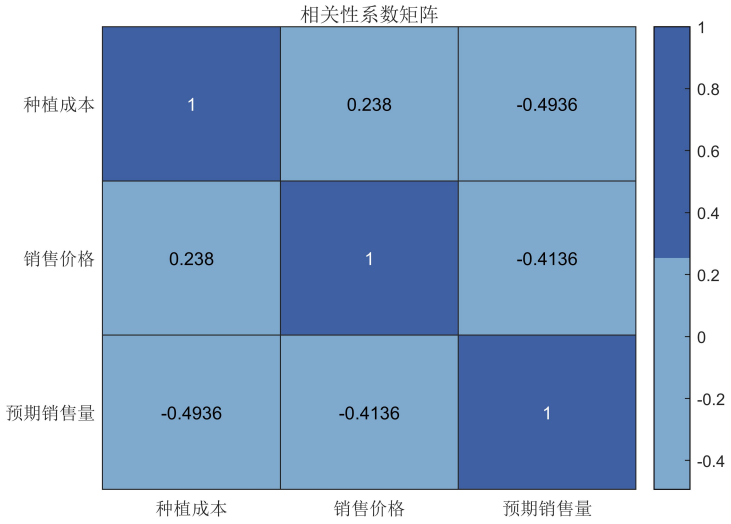


图 **4**　斯皮尔曼相关系数图

由图可知，种植成本和销售价格有很强的正相关性，种植成本与预期销售量呈负相关现象。

2. 曲线拟合

首先，对需要进行分析的数据进行整理与可视化，为后续数据相关性的挖掘打下基础。

经过图 5 观察可以发现，在一些阴显的峰值，数据之间存在着正向与方向的相关关系，下面通过定量分析获得以对数据间的相关性获得更加具体的认识。

由图 4 得到，种植成本和销售价格与预期销售量具有相关性。所以本研究首先打算直接把种植成本与另外两个统计量进行函数拟合，去观察拟合优度如何。

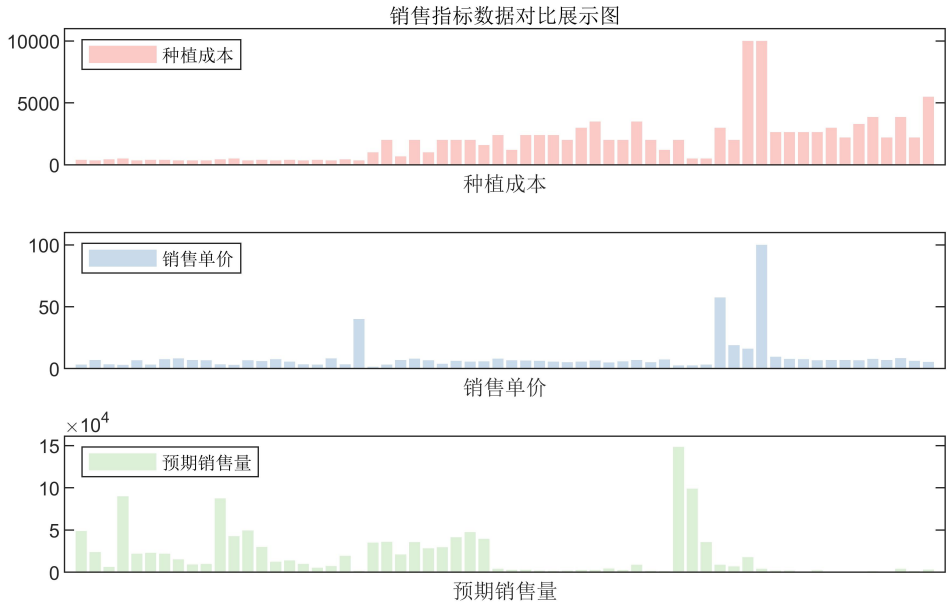


图 **5**

由于三个统计量都属于经济数据，一般都具有异方差性，所以选择对数变化进行函数拟合，使得数据的方差更为稳定，提高拟合的准确性。同时，经过实际拟合证明对数变化后拟合优度确实高于线性回归。直接进行对数拟合的示意图如图 6 所示。

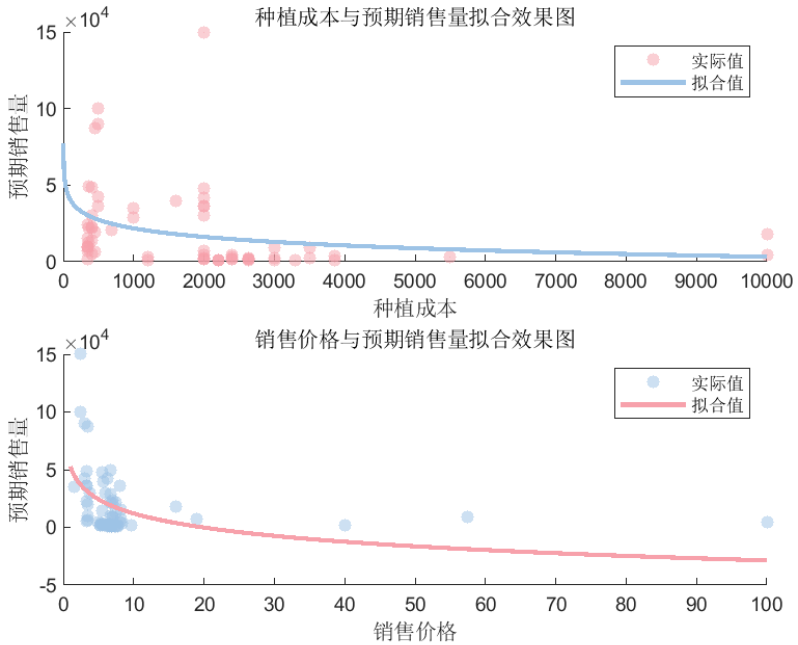


图 **6**　直接拟合效果图

虽然拟合效果相较于线性回归拟合好很多，但是相对于真实数据来说，直接进行对

数拟合优度还是很低。种植成本与销售量拟合的曲线优度 R^2 只有 0.0777，销售单价与预期销售量拟合曲线的 R^2 也只有 0.1829。R^2 的值都过小，拟合效果并不理想。

3. 聚类分析与分组曲线拟合

经过数据观察以及数据检验，发现极有可能是某些农作物性质区别过大引起的。由于各类农作物的实际使用场景差异与稀缺程度差异等，各类农作物说对应的销售与成本等相关指标间存在明显差异。例如，小麦作为日常生活的必需品被广泛地种植，销售价格与种植成本等都相对较低；同时，羊肚菌作为高营养补品，其生长特性使得其并不适合在各个地区进行广泛种植，且极高的营养含量也使得其销售价格与成本等较高。此外，由于两者应用场景的差异与需要程度的不同，两者的预期销售量也存在明显的不同。而这些数据上的差异都会降低直接对部数据与对象进行拟合的效果。因此，本文选择先进行聚类分析的方式，区分不同种类的农作物，再分别进行拟合操作。

为了对农作物进行分类，首先采用了系统聚类方法来确定聚类中心个数，再使用 K-means++ 算法以种植成本和销售单价为依据对农产品进行聚类分析。K-means++ 相对于 K-means 算法来说，解决了对于初值敏感与对于孤立点数据敏感的问题，从而达到更好的聚类效果。

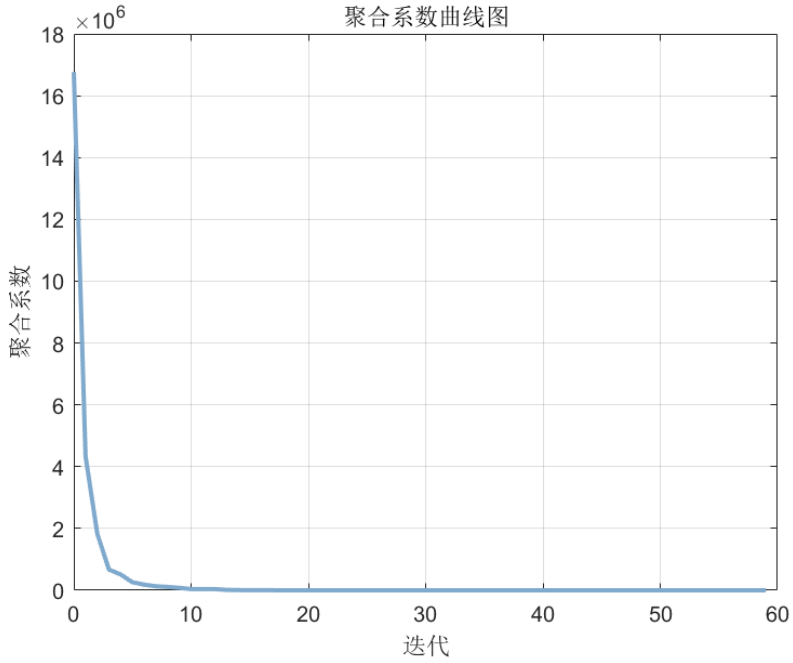


图 **7**

如图 7 所示，系统聚合系数不断迭代优化，到第 5 次的时候聚合系数趋于稳定，可以停止迭代。此时，通过支撑材料中聚合谱系图中可以看出，第五次迭代得到了三个聚类中心，即把农作物分成了三类。

相关的 matlab 求解得到作物分类结果置于支撑材料中中，分类后发现其中第三类农作物只有两种，作物编号分别是 40 和 41。这两种作物的显著特征是种植成本特别高，

达到了 10000 元/亩。第一类农作物的种植成本较高，第二类农作物的种植成本偏低。因为第三类作物较少，种植成本接近第一类，且由于仅有两个点，无法进行后续的曲线拟合分析, 所以我们将第三类农作物并进第一类以简化模型。

所以，最终将农作物分成了两类种植成本较高的记为 I 类，较低的记为 II 类。对这两类植物分别进行成本与销售量等相关曲线的拟合。拟合效果很好，单对于销售单价与销售量而言，两类作物的拟合优度 R 方相比于直接拟合分别提高了 144.1%,32.9%。图 8 是分类之后的相关曲线拟合图。

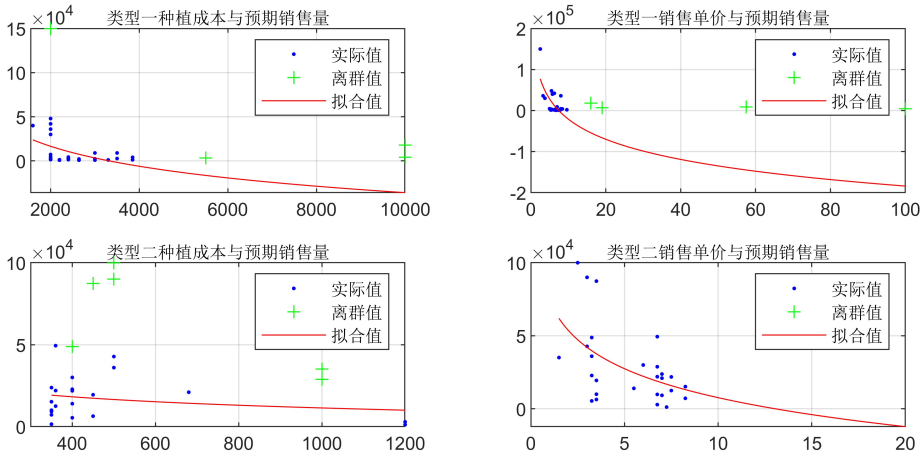


图 **8**　聚类后农作物相关统计量曲线拟合图

4. 模型建立求解

得到作物成本，单价和销量的之间的关系后。为了在问题二的基础上综合考虑相关因素，需保持农作物成本与问题二使用的成本保持一致。因为成本属于比较稳定的因素，变动较小。此时通过拟合好的函数可以计算出农作物该成本下对应的单价和销量。此时，所建立的模型相较于第二问，单价和销量受函数关系影响发生了改变。同时，为了避免销量过剩，约束条件中使得销量总满足小于等于预期销量。此时，输入数据，用 Python 进行模型求解即可得到 2024-2030 年农作物的最优种植方案。

5. 比较分析

为简化计算与数据复杂度，此处仅列出第二问和第三问分别所得到的 2024 年在 34 个地块所做的农作物最优种植策略进行比较。具体数值如表 6 所示。

表 **6**　关于第二问与第三问在 **2024** 年地块种植策略的比较

地块名称	A1	A2	A3	A4	A5	A6	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	C1	C2	C3	C4	C5	C6	总收益/元
第三问	11	2	5	8	7	11	3	9	7	4	1	13	9	11	12	4	13	10	1	7	10	13	9	1	10	2	3107905
第二问	14	12	2	8	5	10	15	9	13	9	15	3	12	4	8	9	14	12	8	2	2	11	7	4	10	8	2823380

第三问通过挖掘各个指标之间的相关性得到拟合函数，以种地成本对预期销售量进

行估计，便于对产量的把控，避免出现供给远超需求的情况，避免了产品浪费，也使得销售利润提升。

比较由第二问第三问的相关利润数据可以清楚地观察到，在将指标相关性纳入考虑后，产品总收益的提升，且各作物产量与预期销售量的差异减小，相当程度上降低了产品滞销问题发生的可能。

六、 模型评价与推广

6.1 模型评价

6.1.1 模型优点

1. 对于不确定参数的设定，本文采用了鲁棒优化为主体的参数确定方式，将预期情况控制在最不理想的状态。这在一定程度上为农民的生产提供了保障，降低了种植产业本身作为一个高风险产业所蕴含的不确定性对种植个体与乡村集体带来巨大损失的概率，为种植产业的发展提供保障。

2. 本文采用遗传算法对求解最优方案，相比于一般优化算法具有更强的全局搜索性，避免陷入局部最优解的循环。

3. 本文采用聚类分析对农作物进行分类，再结合半对数拟合，再一定程度上优化了拟合优度与参数的准确性，且半对数拟合的表述也更为符合经济增长规律。

6.1.2 模型缺陷

1. 在地块面积分配方面，参考 2023 的种植策略，将单一地块的种植作物种类与面积分配严格限制以简化算法，忽略了其他不均匀的种植策略，使得模型应用的灵活性较低。

2. 第二问所采用的鲁棒优化相对于传统优化方式，是一种硬性约束，得到方案相对保守而非绝对的最优解。

3. 第三问仅用预期销售量和销售价格、种植成本进行一对一半对数回归，可能忽略变量之间的多重共线性与内生性问题，导致拟合优度与参数估计情况不佳。

6.2 模型推广

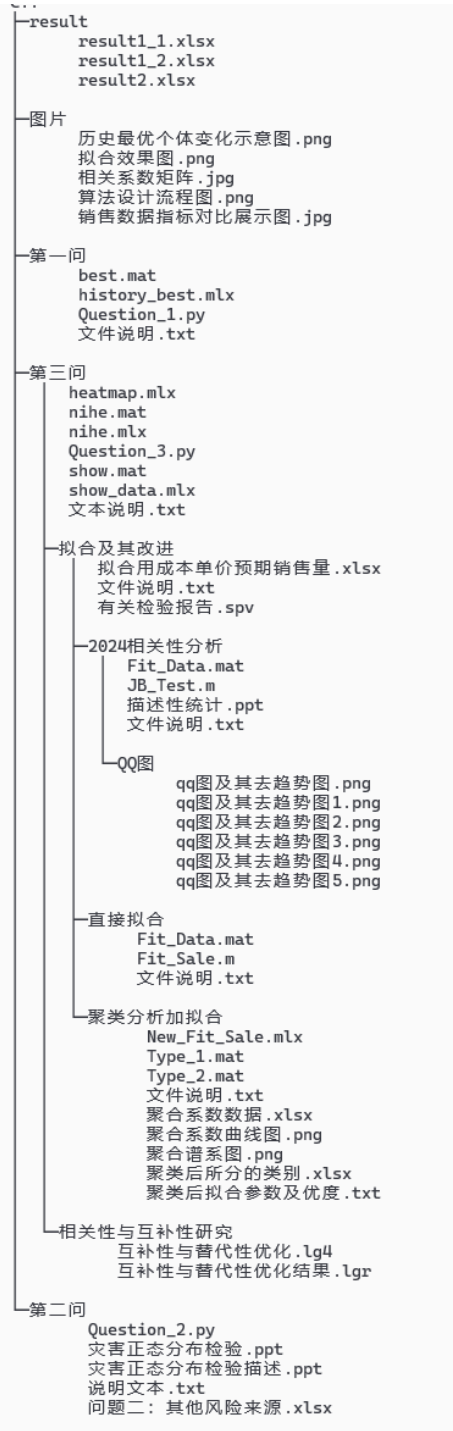
本模型基于启发式算法中的遗传算法更好的克服了陷入种植策略的局部最优解的情况。并且，在推广能力上，综合考虑了农作物种植过程中轮茬、豆科间茬种植、集约化生产、各参量宏观趋势、市场调节因素与农作物本身替代性与互补性的问题。领域跨度大，包含数学统计、智能算法、农业生产与微观经济等多领域综合考量，具有很强的

推广与应用能力，为提高农业生产率，科学改善农民民生提供新路径。也可结合相关政治因素，为保障我国粮食安全提供新思路。

参考文献

[1]《中国统计年鉴 2023》编委会和编辑出版人员 《中国统计年鉴 2023》编委会和编辑出版人员. 付凌晖, 叶礼奇总编, 中国统计年鉴, 中国统计出版社,2023,4-5, 年鉴.

附录 A　支撑材料文件列表



附录 B　问题一相关代码和处理结果

2.1　问题一模型代码

第一问代码，根据遗传算法，分别以目标函数 1 与目标函数 2 最大化为优化方向，计算最优种植策略 >

```
import numpy as np
import random
import pandas as pd

def init_field(*field):
for i in range(len(square)):
field[i][4] = square[i]
for i in range(len(field_type)):
if field_type[i] == "平旱地":
field[i][5] = 1
elif field_type[i] == "梯田":
field[i][5] = 2
elif field_type[i] == "山坡地":
field[i][5] = 3
elif field_type[i] == "水浇地":
field[i][5] = 4
elif field_type[i] == "智慧大棚":
field[i][5] = 6
else:
field[i][5] = 5
return True

def creat(*field):
# 根据约束条件随机生成种植策略
for i in range(26): # 平旱地、梯田、山坡地
if judge_bean(plan_2028[i][0],plan_2029[i][0]):
field[i][0] = random.randint(1, 41)
while judge(field[i][5], field[i][0], plan_2029[i][0], plan_2029[i][1], plan_2029[i][2],
 plan_2029[i][3],1) == False:
field[i][0] = random.randint(1, 41)
else:
field[i][0]=random.choice([1,2,3,4,5])

for i in range(8): # 水浇地
if judge_bean(plan_2028[i+26][0],plan_2028[i+26][1]) or judge_bean(plan_2029[i+26][0],
plan_2029[i+26][1]):
field[i + 26][0] = random.randint(1, 41)
while judge(field[i + 26][5], field[i + 26][0], plan_2029[i + 26][0], plan_2029[i + 26]
[1],plan_2029[i + 26][2], plan_2029[i + 26][3], 0) == False:
field[i + 26][0] = random.randint(1, 41)
```

```
else:
field[i + 26][0] = random.choice([17,18,19])
if field[i + 26][0] == 16: # 为水稻单季种植
continue
else:
for j in range(3):
field[i + 26][j + 1] = random.randint(1, 41)
season = 1
if j >= 1:
season = 2
while judge(field[i + 26][5], field[i + 26][j + 1], plan_2029[i + 26][0], plan_2029[i +
26][1],plan_2029[i + 26][2], plan_2029[i + 26][3], season) == False:
field[i + 26][j + 1] = random.randint(1, 41)

for i in range(16): # 普通大棚
for j in range(4):
season = 1
if j >= 2:
season = 2
if judge_bean(plan_2028[i + 26 + 8][0],plan_2028[i + 26 + 8][1]) or judge_bean(plan_2029
[i + 26 + 8][0],plan_2029[i + 26 + 8][1]):
field[i + 26 + 8][j] = random.randint(1, 41)
while judge(field[i + 26 + 8][5], field[i + 26 + 8][j], plan_2029[i + 26 + 8][0], plan_
2029[i + 26 + 8][1],plan_2029[i + 26 + 8][2], plan_2029[i + 26 + 8][3], season) == False:
field[i + 26 + 8][j] = random.randint(1, 41)
else:
field[i + 26 + 8][j]=random.choice([17,18,19])

for i in range(4): # 智能大棚
for j in range(4):
season = 1
if j >= 2:
season = 2
if judge_bean(plan_2028[i + 26 + 8 + 16][0],plan_2028[i + 26 + 8 + 16][1]) or judge_bea
n(plan_2028[i + 26 + 8 + 16][2],plan_2028[i + 26 + 8 + 16][3]) or judge_bean(plan_2029[
i + 26 + 8 + 16][0],plan_2029[i + 26 + 8 + 16][1]) or judge_bean(plan_2029[i + 26 + 8
+ 16][2],plan_2029[i + 26 + 8 + 16][3]) :
field[i + 26 + 8 + 16][j] = random.randint(1, 41)
else:
field[i + 26 + 8 + 16][j] = random.choice([17, 18, 19])
if season == 1:
while judge(field[i + 26 + 8 + 16][5], field[i + 26 + 8 + 16][j], plan_2029[i + 26 + 8
+ 16][0],plan_2029[i + 26 + 8 + 16][1], plan_2029[i + 26 + 8 + 16][2], plan_2029[i + 2
6 + 8 + 16][3],season) == False:
field[i + 26 + 8 + 16][j] = random.randint(1, 41)
else:
while judge(field[i + 26 + 8 + 16][5], field[i + 26 + 8 + 16][j], plan_2029[i + 26 + 8
```

```
 + 16][0],plan_2029[i + 26 + 8 + 16][1], plan_2029[i + 26 + 8 + 16][2], plan_2029[i + 2
 6 + 8 + 16][3],season) == False or judge(field[i + 26 + 8 + 16][5], field[i + 26 + 8 +
 16][j],field[i + 26 + 8 + 16][0], field[i + 26 + 8 + 16][1],plan_2029[i + 26 + 8 + 16]
 [2], plan_2029[i + 26 + 8 + 16][3],season) == False:
field[i + 26 + 8 + 16][j] = random.randint(1, 41)
return True

def cost(* field):
#print(cost_2023)
cost_sum=0
for i in range(54):
x=int(field[i][0])
y=int(field[i][5]-1)
if field[i][1]==0: # 单季产品
cost_sum+=cost_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]
else:
cost_sum+=cost_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*0.5+cost_2023[int
(field[i][1]-1)][int(field[i][5]-1)]*field[i][4]*0.5+cost_2023[int(field[i][2]-1)][int(fi
eld[i][5]-1)]*field[i][4]*0.5+cost_2023[int(field[i][3]-1)][int(field[i][5]-1)]*field[i][4]*0.5
return cost_sum

def profit_cal(target,* field):
profit_sum=0
if target==1:
for i in range(54):
x=int(field[i][0]-1)
y=int(field[i][5]-1)
if x<=16:
if field[i][1]==0:
profit_sum+=yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*random.uniform(p
rice_2023[int(field[i][0]-1)][int(field[i][5]-1)][0],price_2023[int(field[i][0]-1)][int(fi
eld[i][5]-1)][1])
else:
for k in range(4):
profit_sum+=yield_2023[int(field[i][k]-1)][int(field[i][5]-1)]*field[i][4]*random.uniform(p
rice_2023[int(field[i][k]-1)][int(field[i][5]-1)][0],price_2023[int(field[i][k]-1)][int(fie
ld[i][5]-1)][1])*0.5
else:
if field[i][1]==0:
profit_sum+=yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*(1-0.0108)*random
.uniform(price_2023[int(field[i][0]-1)][int(field[i][5]-1)][0],price_2023[int(field[i][0]-1)
][int(field[i][5]-1)][1])
else:
for k in range(4):
profit_sum+=yield_2023[int(field[i][k]-1)][int(field[i][5]-1)]*field[i][4]*(1-0.0108)*random
.uniform(price_2023[int(field[i][k]-1)][int(field[i][5]-1)][0],price_2023[int(field[i][k]-1)]
[int(field[i][5]-1)][1])*0.5
```

```
return profit_sum

#if sale_2023[int(field[i][0]-1)][int(field[i][5]-1)]<yield_2023[int(field[i][0]-1)][int
(field[i][5]-1)]*field[i][4]:
#profit_sum+=sale_2023[int(field[i][0]-1)][int(field[i][5]-1)]*random.uniform(price_2023
[int(field[i][0]-1)][int(field[i][5]-1)][0],price_2023[int(field[i][0]-1)][int(field[i][5]-1)][1])
#else:
#profit_sum+=yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*random.uniform(price_
2023[int(field[i][0]-1)][int(field[i][5]-1)][0],price_2023[int(field[i][0]-1)][int(field[i][5]-1)][1])

else:
for i in range(54):
x=int(field[i][0]-1)
y=int(field[i][5]-1)
if x<=16:
if field[i][1]==0:
profit_sum+=yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*random.uniform
(price_2023[int(field[i][0]-1)][int(field[i][5]-1)][0],price_2023[int(field[i][0]-1)]
[int(field[i][5]-1)][1])
else:
for k in range(4):
profit_sum+=yield_2023[int(field[i][k]-1)][int(field[i][5]-1)]*field[i][4]*random.uniform
(price_2023[int(field[i][k]-1)][int(field[i][5]-1)][0],price_2023[int(field[i][k]-1)][int
(field[i][5]-1)][1])*0.5
else:
if field[i][1]==0:
profit_sum+=(yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*(1-0.0108)+
yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*0.0108*0.5)*random.uniform
(price_2023[int(field[i][0]-1)][int(field[i][5]-1)][0],price_2023[int(field[i][0]-1)][int(field[i][5]-1)][1])
else:
for k in range(4):
profit_sum+=(yield_2023[int(field[i][k]-1)][int(field[i][5]-1)]*field[i][4]*(1-0.0108)
+yield_2023[int(field[i][k]-1)][int(field[i][5]-1)]*field[i][4]*0.0108*0.5)*random.
uniform(price_2023[int(field[i][k]-1)][int(field[i][5]-1)][0],price_2023[int(field[i]
[k]-1)][int(field[i][5]-1)][1])*0.5
return profit_sum

#if sale_2023[int(field[i][0]-1)][int(field[i][5]-1)]<yield_2023[int(field[i][0]-1)]
[int(field[i][5]-1)]*field[i][4]:
#profit_sum+=sale_2023[int(field[i][0]-1)][int(field[i][5]-1)]*random.uniform(price_2023
[int(field[i][0]-1)][int(field[i][5]-1)][0],price_2023[int(field[i][0]-1)][int(field[i]
[5]-1)][1])+(yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]-sale_2023
[int(field[i][0]-1)][int(field[i][5]-1)])*0.5*random.uniform(price_2023[int(field[i][0]-1)]
[int(field[i][5]-1)][0],price_2023[int(field[i][0]-1)][int(field[i][5]-1)][1])
#else:
#profit_sum+=yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*random.
uniform(price_2023[int(field[i][0]-1)][int(field[i][5]-1)][0],price_2023[int(field[i][0]-1)][int(field[i][5]-1)
```

```
def target_1(*field):
result=profit_cal(1,*field)-cost(*field)
return result

def target_2(*field):
result = profit_cal(2,*field) - cost(*field)
return result

def judge(field_input,crop_input,crop_pre_1,crop_pre_2,crop_pre_3,crop_pre_4,crop_season):
    #判断该地块是否可以种植该作物
enable_123=[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]
enable_4_1=[17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34]
enable_4_2=[35,36,37]
enable_4_special=16
enable_5_1=[17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34]
enable_5_2=[38,39,40,41]
enable_6=[17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34]
#判断平旱地、梯田、山坡地种植情况
if field_input in [1,2,3]:
if crop_input in enable_123 and crop_input != crop_pre_1:
return True
else:
return False
elif field_input == 4:
if crop_season==0 and crop_input==enable_4_special:
if crop_input != crop_pre_1:
return True
else:
return False
else:
if crop_season==1 or crop_season==0:
if crop_input in enable_4_1 and ((crop_input !=crop_pre_1 and crop_input !=crop_pre_2)
 and (crop_input !=crop_pre_3 and crop_input !=crop_pre_4)):
return True
else:
return False
else:
if crop_input in enable_4_2 and ((crop_input !=crop_pre_1 and crop_input !=crop_pre_2)
 and (crop_input !=crop_pre_3 and crop_input !=crop_pre_4)):
return True
else:
return False
elif field_input == 5:
if crop_season == 1:
```

```
if crop_input in enable_5_1 and ((crop_input != crop_pre_1 and crop_input != crop_pre_2)
 and (crop_input != crop_pre_3 and crop_input != crop_pre_4)):
return True
else:
return False
else:
if crop_input in enable_5_2 and ((crop_input != crop_pre_1 and crop_input != crop_pre_2)
and (crop_input != crop_pre_3 and crop_input != crop_pre_4)):
return True
else:
return False
elif field_input == 6:
    #注意使用field_input==6时，crop_pre_1和crop_pre_2需要替换为上一季度的值再进行一次judge
if crop_season == 1:
if crop_input in enable_6 and ((crop_input != crop_pre_1 and crop_input != crop_pre_2)
and (crop_input != crop_pre_3 and crop_input != crop_pre_4)):
return True
else:
return False
else:
if crop_input in enable_6 and ((crop_input != crop_pre_1 and crop_input != crop_pre_2)
 and (crop_input != crop_pre_3 and crop_input != crop_pre_4)):
return True
else:
return False
else:
return False


def judge_bean(crop_pre_1,crop_pre_2):
bean_type=[1,2,3,4,5,17,18,19]
if (crop_pre_1 not in bean_type) and (crop_pre_2 not in bean_type):
return False
else:
return True


#创建地块矩阵 field
#简化为平旱地、梯田、山坡地仅种植一种，水浇地、普通大棚、智慧大棚最多平分种植两种作物
file1=pd.read_excel(r"D:\STUDY\数模国赛2024\C题\附件1.xlsx",sheet_name="乡村的现有耕地")
#print(file1)
square=file1["地块面积/亩"]
field_type=file1["地块类型"]
#print(field_type)
field=np.zeros((54,6))
    #第一季度作物上半（单季），第一季度作物下半，第二季度作物上半，第二季度作物下半，
地块面积，地块类型 1.平旱地 2.梯田 3.山坡地 4.水浇地 5.普通大棚 6.智慧大棚
```

```
field1=np.zeros((54,6))
field2=np.zeros((54,6))
field3=np.zeros((54,6))
field4=np.zeros((54,6))
field5=np.zeros((54,6))
fBest=np.zeros((54,6)) #用于存储最优解

init_field(*field)
init_field(*field1)
init_field(*field2)
init_field(*field3)
init_field(*field4)
init_field(*field5)

#print(field)
#print(field1)

#创建作物矩阵 crop
file2=pd.read_excel(r"D:\STUDY\数模国赛2024\C题\附件1.xlsx",sheet_name="乡村种植的农作物")
crop=np.zeros((41,2)) # 作物编号,作物类型 1.粮食（豆类） 2.粮食 3.蔬菜（豆类） 4.蔬菜 5.食用菌

crop_number=file2["作物编号"]
#print(crop_number)
crop_kind=file2["作物类型"]
for i in range(41):
crop[i][0]=crop_number[i]
for i in range(41):
if crop_kind[i] == "粮食（豆类）":
crop[i][1]=1
elif crop_kind[i] == "粮食":
crop[i][1]=2
elif crop_kind[i] == "蔬菜（豆类）":
crop[i][1]=3
elif crop_kind[i] == "蔬菜":
crop[i][1]=4
else:
crop[i][1]=5
#print(crop)
#result1_1
# plan_2029=[[ 1,0,0,0],[ 4, 0, 0, 0],[ 7, 0, 0, 0],[ 3, 0, 0, 0],[11, 0, 0, 0],[ 5, 0, 0, 0],
#           [ 2, 0, 0, 0],[ 8, 0, 0, 0],[ 3, 0, 0, 0],[14, 0, 0, 0],[ 3, 0, 0, 0],[ 1, 0, 0, 0],
#           [13, 0, 0, 0],[ 3, 0, 0, 0],[ 8, 0, 0, 0],[ 2, 0, 0, 0],[ 3, 0, 0, 0],[ 4, 0, 0, 0],
#           [ 4, 0, 0, 0],[ 7, 0, 0, 0],[ 5, 0, 0, 0],[ 6, 0, 0, 0],[13, 0, 0, 0],[12, 0, 0, 0],
#           [ 2, 0, 0, 0],[12, 0, 0, 0],[17,31,37,37],[17,23,37,37],[17,29,35,37],[29,32,35,35],
#           [23,32,36,35],[28,23,36,37],[34,23,36,37],[17,18,35,36],[29,34,41,40],[19,18,17,19],
#           [18,19,19,17],[18,18,17,19],[18,17,18,18],[19,17,17,18],[32,21,38,38],[18,19,19,19],
#           [18,17,17,19],[17,18,17,17],[22,26,38,40],[30,32,38,41],[18,17,17,17],[21,20,38,41],
```

```
#          [34,33,38,38],[18,27,38,38],[27,21,19,33],[17,17,19,27],[32,32,21,31],[34,31,33,28]]
#
#
# plan_2028=[[14,0,0,0],[10, 0, 0, 0],[ 4, 0, 0, 0],[12, 0, 0, 0],[14, 0, 0, 0],[ 9, 0, 0, 0],
#          [ 4, 0, 0, 0],[ 3, 0, 0, 0],[ 1, 0, 0, 0],[ 9, 0, 0, 0],[15, 0, 0, 0],[13, 0, 0, 0],
#          [ 6, 0, 0, 0],[12, 0, 0, 0],[14, 0, 0, 0],[12, 0, 0, 0],[ 6, 0, 0, 0],[13, 0, 0, 0],
#          [ 2, 0, 0, 0],[ 1, 0, 0, 0],[12, 0, 0, 0],[ 8, 0, 0, 0],[ 1, 0, 0, 0],[ 2, 0, 0, 0],
#          [12, 0, 0, 0],[ 2, 0, 0, 0],[18,29,36,36],[32,21,35,35],[19,19,36,36],[34,23,37,37],
#          [19,24,37,37],[18,20,35,35],[17,22,35,35],[32,23,23,34],[33,31,39,38],[25,25,41,41],
#          [23,23,41,40],[30,33,41,41],[27,32,41,41],[20,29,31,19],[23,23,40,40],[27,32,25,17],
#          [25,24,39,40],[23,22,38,41],[18,25,27,30],[24,31,40,40],[24,25,39,40],[28,22,22,24],
#          [28,27,40,40],[17,21,41,40],[30,26,22,17],[23,23,30,21],[23,17,34,33],[23,18,29,32]]
#
#
# plan_2027=[[ 9, 0, 0, 0],[13, 0, 0, 0],[ 7, 0, 0, 0],[11, 0, 0, 0],[ 2, 0, 0, 0],[14, 0, 0,
    0],
#          [12, 0, 0, 0],[ 2, 0, 0, 0],[ 4, 0, 0, 0],[ 3, 0, 0, 0],[ 9, 0, 0, 0],[ 9, 0, 0, 0],
#          [ 3, 0, 0, 0],[ 4, 0, 0, 0],[ 2, 0, 0, 0],[10, 0, 0, 0],[ 4, 0, 0, 0],[ 8, 0, 0, 0],
#          [ 8, 0, 0, 0],[12, 0, 0, 0],[15, 0, 0, 0],[ 4, 0, 0, 0],[ 6, 0, 0, 0],[10, 0, 0, 0],
#          [ 9, 0, 0, 0],[10, 0, 0, 0],[22,25,35,37],[30,20,37,36],[17,22,37,35],[21,17,36,35],
#          [34,25,36,35],[32,23,37,37],[19,28,37,36],[27,24,23,29],[17,18,19,17],[34,29,39,38],
#          [20,28,39,39],[25,34,39,40],[24,25,39,38],[34,25,38,41],[18,19,19,18],[23,25,39,39],
#          [28,32,41,38],[28,27,39,40],[23,34,40,38],[17,17,19,19],[26,21,38,38],[18,19,17,17],
#          [18,31,38,39],[19,23,39,38],[19,32,33,23],[26,27,30,28],[18,30,20,25],[20,25,27,24]]
#
# plan_2026=[[ 6,0,0,0],[ 3,0,0,0],[13,0,0,0],[ 2,0,0,0],[11,0,0,0],[ 2,0,0,0],
#          [ 6,0,0,0],[ 1,0,0,0],[11,0,0,0],[ 2,0,0,0],[ 1,0,0,0],[ 3,0,0,0],
#          [15,0,0,0],[11,0,0,0],[13,0,0,0],[ 4,0,0,0],[ 3,0,0,0],[ 2,0,0,0],
#          [ 7,0,0,0],[ 9,0,0,0],[ 5,0,0,0],[12,0,0,0],[12,0,0,0],[ 9,0,0,0],
#          [ 4,0,0,0],[15,0,0,0],[31,8,5,7],[27,4,6,6],[32,9,7,7],[31,0,7,7],
#          [17,3,5,7],[17,9,7,6],[29,0,5,6],[18,5,6,6],[33,9,7,8],[19,8,0,9],
#          [24,4,9,2],[18,8,8,8],[17,5,9,1],[23,9,8,0],[27,8,0,3],[22,3,4,2],
#          [18,7,7,9],[17,7,9,9],[24,0,0,8],[25,7,9,1],[19,8,9,8],[32,0,9,9],
#          [19,7,8,9],[18,9,9,7],[17,7,8,2],[18,9,7,3],[29,8,0,5],[30,7,1,4]]
#
# plan_2025=[[5,0,0,0],[10, 0, 0, 0],[ 5, 0, 0, 0],[ 8, 0, 0, 0],[15, 0, 0, 0],[11, 0, 0, 0],
#          [ 1, 0, 0, 0],[12, 0, 0, 0],[ 7, 0, 0, 0],[ 8, 0, 0, 0],[ 7, 0, 0, 0],[11, 0, 0, 0],
#          [ 2, 0, 0, 0],[ 5, 0, 0, 0],[ 6, 0, 0, 0],[10, 0, 0, 0],[ 5, 0, 0, 0],[14, 0, 0, 0],
#          [ 4, 0, 0, 0],[ 1, 0, 0, 0],[15, 0, 0, 0],[10, 0, 0, 0],[ 2, 0, 0, 0],[ 2, 0, 0, 0],
#          [ 7, 0, 0, 0],[ 3, 0, 0, 0],[29,19,36,36],[33,19,37,35],[16,21,35,36],[19,32,35,35],
#          [27,26,36,36],[24,29,35,35],[27,23,37,37],[32,21,35,37],[26,32,41,41],[21,21,41,38],
#        [17,27,41,38],[21,32,39,38],[23,27,40,38],[18,34,39,41],[27,23,40,38],[27,29,40,39],
#        [32,21,41,41],[21,18,38,41],[18,30,41,41],[22,34,40,40],[24,21,39,39],[24,23,41,41],
#        [20,28,39,39],[32,29,39,41],[19,22,34,29],[25,26,22,22],[24,31,21,21],[27,27,26,25]]
#
#
# plan_2024=[[2,0,0,0],[15, 0, 0, 0],[ 8, 0, 0, 0],[ 9, 0, 0, 0],[10, 0, 0, 0],[14, 0, 0, 0],
```

```
#          [ 3, 0, 0, 0],[ 6, 0, 0, 0],[ 2, 0, 0, 0],[13, 0, 0, 0],[ 8, 0, 0, 0],[ 1, 0, 0, 0],
#          [ 1, 0, 0, 0],[ 2, 0, 0, 0],[ 2, 0, 0, 0],[ 6, 0, 0, 0],[ 4, 0, 0, 0],[11, 0, 0, 0],
#          [ 3, 0, 0, 0],[ 6, 0, 0, 0],[ 6, 0, 0, 0],[ 3, 0, 0, 0],[ 4, 0, 0, 0],[ 7, 0, 0, 0],
#          [12, 0, 0, 0],[12, 0, 0, 0],[18,22,37,35],[25,23,36,36],[23,19,37,37],[29,26,37,36],
#          [25,21,37,35],[26,34,36,36],[30,17,36,36],[28,29,36,36],[33,31,39,40],[22,17,39,40],
#          [18,28,39,40],[22,30,41,40],[34,18,41,41],[25,31,40,38],[22,24,39,41],[34,24,38,41],
#          [23,27,39,39],[27,23,40,39],[33,22,38,40],[19,33,38,39],[28,34,38,40],[17,31,40,38],
#          [29,32,38,38],[28,27,38,38],[27,30,31,17],[33,23,34,30],[33,19,18,29],[29,31,18,20]]

#2023的种植情况，按照field土地顺序

#result1_2
plan_2029=[[4,0,0,0],[ 3, 0, 0, 0],[10, 0, 0, 0],[ 5, 0, 0, 0],[ 2, 0, 0, 0],[11, 0, 0, 0],
[14, 0, 0, 0],[12, 0, 0, 0],[ 2, 0, 0, 0],[12, 0, 0, 0],[13, 0, 0, 0],[ 8, 0, 0, 0],[ 6, 0, 0,
    0],
[13, 0, 0, 0],[ 3, 0, 0, 0],[10, 0, 0, 0],[ 1, 0, 0, 0],[ 8, 0, 0, 0],[ 9, 0, 0, 0],[ 5, 0, 0,
    0]
,[11, 0, 0, 0],[15, 0, 0, 0],[ 2, 0, 0, 0],[ 6, 0, 0, 0],[ 8, 0, 0, 0],[ 3, 0, 0,
    0],[32,26,37,37],
[30,26,36,35],[29,28,35,35],[27,26,36,36],[22,17,35,37],[17,20,35,37],[34,33,36,37]
,[17,29,35,35],[21,18,41,38],[27,25,38,40],[19,27,18,28],[34,23,38,41],[28,28,39,39],[26,29,39,40],
[20,31,39,41],[27,20,41,41],[33,28,38,39],[20,20,38,38],[18,18,17,19],[32,29,40,38],[33,26,40,41],
[22,29,41,39],[30,28,39,40],[26,27,38,38],[28,19,20,33],[34,25,33,32],[19,18,17,27],[32,26,22,21]]

plan_2028=[[6,0,0,0],[ 2, 0, 0, 0],[ 2, 0, 0, 0],[13, 0, 0, 0],[ 4, 0, 0, 0],[ 1, 0, 0, 0],
[ 1, 0, 0, 0],[ 5, 0, 0, 0],[ 8, 0, 0, 0],[ 4, 0, 0, 0],[12, 0, 0, 0],[ 5, 0, 0, 0],
[ 1, 0, 0, 0],[ 1, 0, 0, 0],[15, 0, 0, 0],[ 4, 0, 0, 0],[ 4, 0, 0, 0],[ 1, 0, 0, 0],
[ 3, 0, 0, 0],[ 7, 0, 0, 0],[ 4, 0, 0, 0],[ 9, 0, 0, 0],[ 4, 0, 0, 0],[13, 0, 0, 0],
[15, 0, 0, 0],[12, 0, 0, 0],[19,29,35,35],[18,33,37,37],[26,21,37,36],[17,32,37,37],
[31,19,36,36],[25,23,36,36],[17,32,35,35],[19,25,37,36],[23,32,40,39],[17,18,19,17],
[18,18,19,17],[19,18,17,19],[18,17,38,41],[32,17,41,41],[18,19,17,18],[19,18,19,19],
[18,17,19,17],[32,22,41,39],[31,29,40,38],[18,17,39,39],[19,18,19,19],[18,17,18,18],
[17,17,17,18],[18,30,41,39],[23,27,17,34],[21,20,24,29],[30,34,22,29],[28,28,30,19]]



plan_2027=[[ 2,0,0,0],[11, 0, 0, 0],[12, 0, 0, 0],[ 9, 0, 0, 0],[13, 0, 0, 0],[ 2, 0, 0, 0],
[ 3, 0, 0, 0],[12, 0, 0, 0],[ 1, 0, 0, 0],[ 8, 0, 0, 0],[ 4, 0, 0, 0],[ 4, 0, 0, 0],
[ 2, 0, 0, 0],[ 7, 0, 0, 0],[ 2, 0, 0, 0],[13, 0, 0, 0],[11, 0, 0, 0],[14, 0, 0, 0],
[ 4, 0, 0, 0],[ 9, 0, 0, 0],[ 9, 0, 0, 0],[ 4, 0, 0, 0],[ 5, 0, 0, 0],[ 4, 0, 0, 0],
[ 3, 0, 0, 0],[ 6, 0, 0, 0],[16,28,37,36],[17,22,35,35],[18,30,35,35],[20,23,36,35],
[34,20,35,35],[29,33,35,35],[29,34,36,37],[18,22,35,35],[17,18,19,19],[22,25,31,20],
[33,32,39,41],[25,23,38,41],[34,26,40,39],[33,34,39,40],[33,25,20,29],[32,26,40,38],
[23,28,40,39],[18,17,19,17],[30,28,39,39],[20,28,40,38],[32,25,40,39],[26,22,41,41],
[34,22,17,22],[22,17,38,40],[22,26,19,29],[19,32,31,27],[27,27,31,31],[17,17,24,18]]

plan_2026=[[ 3,0,0,0],[13, 0, 0, 0],[ 4, 0, 0, 0],[ 2, 0, 0, 0],[ 5, 0, 0, 0],[ 1, 0, 0, 0],
[ 2, 0, 0, 0],[13, 0, 0, 0],[ 3, 0, 0, 0],[12, 0, 0, 0],[11, 0, 0, 0],[11, 0, 0, 0],
```

```
[15, 0, 0, 0],[13, 0, 0, 0],[14, 0, 0, 0],[ 8, 0, 0, 0],[ 2, 0, 0, 0],[13, 0, 0, 0],
[14, 0, 0, 0],[ 3, 0, 0, 0],[ 7, 0, 0, 0],[ 1, 0, 0, 0],[ 9, 0, 0, 0],[10, 0, 0, 0],
[12, 0, 0, 0],[ 5, 0, 0, 0],[30,29,35,35],[23,32,37,37],[26,29,37,36],[33,26,37,37],
[17,27,37,37],[22,17,36,36],[20,32,35,35],[24,28,37,37],[27,20,39,39],[21,31,39,41],
[28,30,38,40],[33,29,40,40],[17,29,21,19],[22,17,41,38],[21,29,17,26],[31,28,41,39],
[26,31,41,38],[31,24,41,39],[31,18,22,34],[26,17,31,17],[28,31,41,38],[32,33,38,39],
[25,32,38,40],[20,34,41,39],[34,20,17,32],[33,24,28,29],[26,18,21,24],[21,28,28,30]]


plan_2025=[[4,0,0,0],[ 2, 0, 0, 0],[15, 0, 0, 0],[ 7, 0, 0, 0],[ 4, 0, 0, 0],[12, 0, 0, 0],
[12, 0, 0, 0],[ 1, 0, 0, 0],[ 9, 0, 0, 0],[ 3, 0, 0, 0],[ 9, 0, 0, 0],[ 5, 0, 0, 0],
[ 1, 0, 0, 0],[ 5, 0, 0, 0],[ 3, 0, 0, 0],[ 5, 0, 0, 0],[ 9, 0, 0, 0],[ 1, 0, 0, 0],
[ 3, 0, 0, 0],[ 9, 0, 0, 0],[ 2, 0, 0, 0],[11, 0, 0, 0],[12, 0, 0, 0],[12, 0, 0, 0],
[12, 0, 0, 0],[ 8, 0, 0, 0],[18,31,36,36],[30,20,29,33],[25,32,35,35],[17,21,36,35],
[34,18,33,33],[32,26,35,37],[17,29,37,36],[29,21,36,36],[30,30,40,40],[17,19,18,19],
[17,19,19,18],[19,18,18,18],[25,27,19,30],[23,29,20,19],[30,17,40,41],[22,18,24,32],
[25,17,40,39],[21,34,33,32],[26,18,41,38],[30,27,29,34],[22,18,26,17],[19,17,19,17],
[21,22,41,39],[17,19,17,19],[18,19,28,33],[18,18,19,17],[31,31,22,23],[23,20,29,28]]


plan_2024=[[ 9,0,0,0],[ 2, 0, 0, 0],[15, 0, 0, 0],[ 6, 0, 0, 0],[11, 0, 0, 0],[11, 0, 0, 0],
[11, 0, 0, 0],[14, 0, 0, 0],[ 8, 0, 0, 0],[ 3, 0, 0, 0],[ 4, 0, 0, 0],[ 6, 0, 0, 0],
[ 1, 0, 0, 0],[11, 0, 0, 0],[ 8, 0, 0, 0],[ 3, 0, 0, 0],[11, 0, 0, 0],[14, 0, 0, 0],
[ 9, 0, 0, 0],[11, 0, 0, 0],[ 6, 0, 0, 0],[11, 0, 0, 0],[ 9, 0, 0, 0],[ 5, 0, 0, 0],
[12, 0, 0, 0],[12, 0, 0, 0],[29,22, 35,37],[22,19, 37,37],[18,29, 36,36],[31,32, 19,27],
[29,18, 37,37],[19,22, 36,36],[33,32, 25,26],[19,17, 37,35],[17,32, 41,39],[25,20, 39,41],
[29,24, 41,41],[27,20, 40,38],[21,31, 29,20],[17,24, 41,41],[26,23, 39,38],[23,32, 39,41],
[27,28, 38,41],[18,18, 39,38],[29,21, 40,39],[31,32, 38,38],[20,19, 40,39],[23,25, 27,26],
[33,17, 38,40],[32,24, 38,38],[31,31, 27,27],[34,24, 28,30],[18,26, 31,23],[18,25, 17,30]]


plan_2023=[[6,0,0,0],[7,0,0,0],[7,0,0,0],[1,0,0,0],[4,0,0,0],[8,0,0,0],
[6,0,0,0],[2,0,0,0],[3,0,0,0],[4,0,0,0],[5,0,0,0],[8,0,0,0],
[6,0,0,0],[8,0,0,0],[9,0,0,0],[10,0,0,0],[1,0,0,0],[7,0,0,0],
[14,0,0,0],[15,0,0,0],[11,0,0,0],[12,0,0,0],[1,0,0,0],[13,0,0,0],
[6,0,0,0],[3,0,0,0],[20,20,36,36],[28,28,35,35],[21,21,35,35],[22,22,35,35],
[17,17,36,36],[18,18,37,37],[16,0,0,0],[16,0,0,0],[18,18,38,38],[24,24,38,38],
[25,25,38,38],[26,26,39,39],[28,28,39,39],[27,27,39,39],[19,19,40,40],[19,19,40,40],
[18,18,40,40],[17,17,41,41],[17,17,41,41],[22,22,41,41],[21,21,41,41],[29,29,41,41],
[30,27,41,41],[31,31,41,41],[32,33,24,21],[25,26,22,29],[17,17,28,30],[19,19,34,23]]

#2023亩产量，产品编号*土地类型
yield_2023=[[400,380,360,0,0,0],[500,475,450,0,0,0],[400,380,360,0,0,0],[350,330,315,0,0,0],
[415,395,375,0,0,0],[800,760,720,0,0,0]
,[100,950,900,0,0,0],[400,380,360,0,0,0],[630,600,570,0,0,0],[525,500,475,0,0,0],[110,105,100,0,0,0],
[3000,2850,27000,0,0]
,[2200,2100,2000,0,0,0],[420,400,380,0,0,0],[525,500,475,0,0,0],[0,0,0,500,0,0],
```

```
[0,0,0,3000,3600,3200],[0,0,0,2000,2400,2200]
,[0,0,0,3000,3600,3200],[0,0,0,2000,2400,2200],[0,0,0,2400,3000,2700],[0,0,0,6400,8000,7200]
,[0,0,0,2700,3300,3000],[0,0,0,2400,3000,2700]
,[0,0,0,3300,4000,3600],[0,0,0,3700,4500,4100],[0,0,0,4100,5000,4500],[0,0,0,3200,4000,3600]
,[0,0,0,12000,15000,13500],[0,0,0,4100,5000,4500]
,[0,0,0,1600,2000,1800],[0,0,0,10000,12000,11000],[0,0,0,5000,6000,5400],
[0,0,0,5500,6600,6000],[0,0,0,5000,0,0],[0,0,0,4000,0,0]
,[0,0,0,3000,0,0],[0,0,0,0,5000,0],[0,0,0,0,4000,0],[0,0,0,0,10000,0],[0,0,0,0,1000,0]]

#2023种植成本，产品编号*土地类型
cost_2023=[[400,400,400,0,0,0],[400,400,400,0,0,0],[350,350,350,0,0,0],[350,350,350,0,0,0]
,[350,350,350,0,0,0],[450,450,450,0,0,0],
[500,500,500,0,0,0],[360,360,360,0,0,0],[400,400,400,0,0,0],[360,360,360,0,0,0]
,[350,350,350,0,0,0],[100,1000,1000,0,0,0],
[200,2000,2000,0,0,0],[400,400,400,0,0,0],[350,350,350,0,0,0],[0,0,0,680,0,0],[0,0,0,2000,2400,2640],[0,0,0,100
[0,0,0,2000,2400,2640],[0,0,0,2000,2400,2640],[0,0,0,2000,2400,2640],[0,0,0,2000,2400,2640]
,[0,0,0,2300,2700,3000],[0,0,0,1600,2000,2200],
[0,0,0,2400,3000,3300],[0,0,0,2900,3500,3850],[0,0,0,1600,2000,2200],[0,0,0,1600,2000,2200]
,[0,0,0,2900,3500,3850],[0,0,0,1600,2000,2200],
[0,0,0,1000,1200,1300],[0,0,0,4100,5000,5500],[0,0,0,2000,2500,2750],[0,0,0,900,1100,1200]
,[0,0,0,2000,0,0],[0,0,0,500,0,0],[0,0,0,500,0,0],
[0,0,0,0,3000,0],[0,0,0,0,2000,0],[0,0,0,0,10000,0],[0,0,0,0,10000,0]]

#2023种植销售单价，产品编号*土地类型
price_2023=[[[2.50,4.00],[2.50,4.00],[2.50,4.00],[0,0],[0,0],[0,0]],
[[6.50,8.50],[6.50,8.50],[6.50,8.50],[0,0],[0,0],[0,0]],
[[7.50,9.00],[7.50,9.00],[7.50,9.00],[0,0],[0,0],[0,0]],
[[6.00,8.00],[6.00,8.00],[6.00,8.00],[0,0],[0,0],[0,0]],
[[6.00,7.50],[6.00,7.50],[6.00,7.50],[0,0],[0,0],[0,0]],
[[3.00,4.00],[3.00,4.00],[3.00,4.00],[0,0],[0,0],[0,0]],
[[2.50,3.50],[2.50,3.50],[2.50,3.50],[0,0],[0,0],[0,0]],
[[6.00,7.50],[6.00,7.50],[6.00,7.50],[0,0],[0,0],[0,0]],
[[5.50,6.50],[5.50,6.50],[5.50,6.50],[0,0],[0,0],[0,0]],
[[6.50,8.50],[6.50,8.50],[6.50,8.50],[0,0],[0,0],[0,0]],
[[30.00,50.00],[30.00,50.00],[30.00,50.00],[0,0],[0,0],[0,0]],
[[1.00,2.00],[1.00,2.00],[1.00,2.00],[0,0],[0,0],[0,0]],
[[2.50,4.00],[2.50,4.00],[2.50,4.00],[0,0],[0,0],[0,0]],
[[5.00,6.00],[5.00,6.00],[5.00,6.00],[0,0],[0,0],[0,0]],
[[3.00,4.00],[3.00,4.00],[3.00,4.00],[0,0],[0,0],[0,0]],
[[0,0],[0,0],[0,0],[6.00,8.00],[0,0],[0,0]],
[[0,0],[0,0],[0,0],[7.00,9.00],[7.00,9.00],[8.40,10.80]],
[[0,0],[0,0],[0,0],[5.50,8.00],[5.50,8.00],[6.60,9.60]],
[[0,0],[0,0],[0,0],[5.00,8.00],[5.00,8.00],[6.00,9.60]],
[[0,0],[0,0],[0,0],[3.00,4.50],[3.00,4.50],[3.60,5.40]],
[[0,0],[0,0],[0,0],[5.00,7.50],[5.00,7.50],[6.00,9.00]],
[[0,0],[0,0],[0,0],[5.00,6.00],[5.00,6.00],[6.00,7.20]],
[[0,0],[0,0],[0,0],[4.80,6.70],[4.80,6.70],[5.80,8.00]],
```

```
[[0,0],[0,0],[0,0],[4.00,6.50],[4.00,6.50],[5.80,7.80]],
[[0,0],[0,0],[0,0],[5.00,6.00],[5.00,6.00],[6.00,7.20]],
[[0,0],[0,0],[0,0],[5.50,7.50],[5.50,7.50],[6.60,9.00]],
[[0,0],[0,0],[0,0],[4.00,6.00],[4.00,6.00],[4.80,7.20]],
[[0,0],[0,0],[0,0],[5.00,6.50],[5.00,6.50],[6.00,7.80]],
[[0,0],[0,0],[0,0],[6.00,8.00],[6.00,8.00],[7.20,9.60]],
[[0,0],[0,0],[0,0],[4.50,6.00],[4.50,6.00],[5.40,7.20]],
[[0,0],[0,0],[0,0],[6.00,8.50],[6.00,8.50],[7.20,10.20]],
[[0,0],[0,0],[0,0],[3.00,6.00],[3.00,6.00],[3.60,7.20]],
[[0,0],[0,0],[0,0],[4.00,5.00],[4.00,5.00],[4.80,6.00]],
[[0,0],[0,0],[0,0],[3.20,4.80],[3.20,4.80],[3.80,5.80]],
[[0,0],[0,0],[0,0],[2.00,3.00],[0,0],[0,0]],
[[0,0],[0,0],[0,0],[2.00,3.00],[0,0],[0,0]],
[[0,0],[0,0],[0,0],[2.50,4.00],[0,0],[0,0]],
[[0,0],[0,0],[0,0],[0,0],[50.00,65.00],[0,0]],
[[0,0],[0,0],[0,0],[0,0],[18.00,20.00],[0,0]],
[[0,0],[0,0],[0,0],[0,0],[14.00,18.00],[0,0]],
[[0,0],[0,0],[0,0],[0,0],[80.00,120.00],[0,0]]]

#遗传算法寻找最优解
#创建种群个体
creat(*field)
creat(*field1)
creat(*field2)
creat(*field3)
creat(*field4)
creat(*field5)
fitness=[0,0,0,0,0,0] #存储各个个体的适应度
Gtime=500#迭代次数
q_possibility=0.7 #发生交叉的概率
p_possibility=0.1 #发生变异的概率
fitBest=0 #记录最优适应度
for i in range(Gtime):
fitness[0] = target_2(*field)
fitness[1] = target_2(*field1)
fitness[2] = target_2(*field2)
fitness[3] = target_2(*field3)
fitness[4] = target_2(*field4)
fitness[5] = target_2(*field5)
maxfit=max(fitness)
minfit=min(fitness)
for j in range(len(fitness)):
if fitness[j]==maxfit and maxfit>fitBest:
fitBest=maxfit
findex=j #存储当前最优解所在位置
for j in range(len(fitness)):
if fitness[j]==minfit:
```

```
fDiscard=j #存储当前最劣解所在位置

if findex==0 and maxfit==fitBest:
fBest=field
elif findex==1 and maxfit==fitBest:
fBest=field1
elif findex==2 and maxfit==fitBest:
fBest=field2
elif findex==3 and maxfit==fitBest:
fBest=field3
elif findex==4 and maxfit==fitBest:
fBest=field4
elif findex == 5 and maxfit == fitBest:
fBest=field5

if fDiscard==0:#舍弃最劣的解，用新的种群个体替换
init_field(*field)
creat(*field)
elif fDiscard==1:
init_field(*field1)
creat(*field1)
elif fDiscard==2:
init_field(*field2)
creat(*field2)
elif fDiscard==3:
init_field(*field3)
creat(*field3)
elif fDiscard==4:
init_field(*field4)
creat(*field4)
else:
init_field(*field5)
creat(*field5)

#交叉操作
q=random.random()
if q<q_possibility:
x=random.randint(0,53)
for m in range(4):
temp=field[x][m]
field[x][m]=field1[x][m]
field1[x][m]=temp
q = random.random()
if q < q_possibility:
x = random.randint(0, 53)
for m in range(4):
temp = field1[x][m]
```

```
field1[x][m] = field2[x][m]
field2[x][m] = temp
q = random.random()
if q < q_possibility:
x = random.randint(0, 53)
for m in range(4):
temp = field2[x][m]
field2[x][m] = field3[x][m]
field3[x][m] = temp
q = random.random()
if q < q_possibility:
x = random.randint(0, 53)
for m in range(4):
temp = field3[x][m]
field3[x][m] = field4[x][m]
field4[x][m] = temp
q = random.random()
if q < q_possibility:
x = random.randint(0, 53)
for m in range(4):
temp = field4[x][m]
field4[x][m] = field5[x][m]
field5[x][m] = temp
#print(fitBest)

#变异操作
p = random.random()
if p < p_possibility:
x = random.randint(0, 53)
if x>26:
for m in range(4):
field[x][m]=random.randint(1,41)
while
    judge(field[x][5],field[x][m],plan_2023[x][0],plan_2023[x][1],plan_2023[x][2],plan_2023[x][3],1)==False:
field[x][m] = random.randint(1, 41)
else:
field[x][0] = random.randint(1, 41)
while judge(field[x][5], field[x][0], plan_2023[x][0], plan_2023[x][1], plan_2023[x][2],
    plan_2023[x][3],1) == False:
field[x][0] = random.randint(1, 41)
if p < p_possibility:
x = random.randint(0, 53)
if x>26:
for m in range(4):
field1[x][m]=random.randint(1,41)
while
    judge(field1[x][5],field1[x][m],plan_2023[x][0],plan_2023[x][1],plan_2023[x][2],plan_2023[x][3],1)==False:
```

```
field1[x][m] = random.randint(1, 41)
else:
field1[x][0] = random.randint(1, 41)
while judge(field1[x][5], field1[x][0], plan_2023[x][0], plan_2023[x][1], plan_2023[x][2],
    plan_2023[x][3],1) == False:
field1[x][0] = random.randint(1, 41)
if p < p_possibility:
x = random.randint(0, 53)
if x>26:
for m in range(4):
field2[x][m]=random.randint(1,41)
while
    judge(field2[x][5],field2[x][m],plan_2023[x][0],plan_2023[x][1],plan_2023[x][2],plan_2023[x][3],1)==False:
field2[x][m] = random.randint(1, 41)
else:
field2[x][0] = random.randint(1, 41)
while judge(field2[x][5], field2[x][0], plan_2023[x][0], plan_2023[x][1], plan_2023[x][2],
    plan_2023[x][3],1) == False:
field2[x][0] = random.randint(1, 41)
if p < p_possibility:
x = random.randint(0, 53)
if x>26:
for m in range(4):
field3[x][m]=random.randint(1,41)
while
    judge(field3[x][5],field3[x][m],plan_2023[x][0],plan_2023[x][1],plan_2023[x][2],plan_2023[x][3],1)==False:
field3[x][m] = random.randint(1, 41)
else:
field3[x][0] = random.randint(1, 41)
while judge(field3[x][5], field3[x][0], plan_2023[x][0], plan_2023[x][1], plan_2023[x][2],
    plan_2023[x][3],1) == False:
field3[x][0] = random.randint(1, 41)
if p < p_possibility:
x = random.randint(0, 53)
if x>26:
for m in range(4):
field4[x][m]=random.randint(1,41)
while
    judge(field4[x][5],field4[x][m],plan_2023[x][0],plan_2023[x][1],plan_2023[x][2],plan_2023[x][3],1)==False:
field4[x][m] = random.randint(1, 41)
else:
field4[x][0] = random.randint(1, 41)
while judge(field4[x][5], field4[x][0], plan_2023[x][0], plan_2023[x][1], plan_2023[x][2],
    plan_2023[x][3],1) == False:
field4[x][0] = random.randint(1, 41)
if p < p_possibility:
x = random.randint(0, 53)
```

```
if x>26:
for m in range(4):
field5[x][m]=random.randint(1,41)
while
    judge(field5[x][5],field5[x][m],plan_2023[x][0],plan_2023[x][1],plan_2023[x][2],plan_2023[x][3],1)==False:
field5[x][m] = random.randint(1, 41)
else:
field5[x][0] = random.randint(1, 41)
while judge(field5[x][5], field5[x][0], plan_2023[x][0], plan_2023[x][1], plan_2023[x][2],
    plan_2023[x][3],1) == False:
field5[x][0] = random.randint(1, 41)
print(fitBest)
print(fBest)
# a=cost(* field)
# print(a)
# a=profit_cal(* field)
# print(a)
# a=target_1(*field)
# print(a)
```

2.2 历史最优个体对应收益变化图绘制代码

第一问遗传算法历史最优个体对应收益变化图绘制代码。

```
load best.mat
x=linspace(0,1,200)
y=best(:,1)
plot(x,y,'LineWidth',2)
title("历史最优个体变化示意图")
legend("历史最优解",'Location','best')
set(gca,'XTick',[])
ylabel("收益")
```

附录 C 问题二相关代码和处理结果

```
import numpy as np
import random
import pandas as pd

def init_field(*field):
for i in range(len(square)):
field[i][4] = square[i]
for i in range(len(field_type)):
if field_type[i] == "平旱地":
```

```
field[i][5] = 1
elif field_type[i] == "梯田":
field[i][5] = 2
elif field_type[i] == "山坡地":
field[i][5] = 3
elif field_type[i] == "水浇地":
field[i][5] = 4
elif field_type[i] == "智慧大棚":
field[i][5] = 6
else:
field[i][5] = 5
return True

def creat(*field):
# 根据约束条件随机生成种植策略
for i in range(26): # 平旱地、梯田、山坡地
if judge_bean(plan_2023[i][0],plan_2024[i][0]):
field[i][0] = random.randint(1, 41)
while judge(field[i][5], field[i][0], plan_2024[i][0], plan_2024[i][1], plan_2024
[i][2], plan_2024[i][3],1) == False:
field[i][0] = random.randint(1, 41)
else:
field[i][0]=random.choice([1,2,3,4,5])
# field[i][0] = random.randint(1, 41)
# while judge(field[i][5], field[i][0], plan_2023[i][0], plan_2023[i][1], plan_2023
[i][2], plan_2023[i][3],1) == False:
#     field[i][0] = random.randint(1, 41)
for i in range(8): # 水浇地
if judge_bean(plan_2023[i+26][0],plan_2023[i+26][1]) or judge_bean(plan_2024[i+26][0]
,plan_2024[i+26][1]):
field[i + 26][0] = random.randint(1, 41)
while judge(field[i + 26][5], field[i + 26][0], plan_2024[i + 26][0], plan_2024[i + 26]
[1],plan_2024[i + 26][2], plan_2024[i + 26][3], 0) == False:
field[i + 26][0] = random.randint(1, 41)
else:
field[i + 26][0] = random.choice([17,18,19])
# field[i + 26][0] = random.randint(1, 41)
# while judge(field[i + 26][5], field[i + 26][0], plan_2023[i + 26][0], plan_2023[i + 26]
[1],plan_2023[i + 26][2], plan_2023[i + 26][3], 0) == False:
#     field[i + 26][0] = random.randint(1, 41)

if field[i + 26][0] == 16: # 为水稻单季种植
continue
else:
for j in range(3):
field[i + 26][j + 1] = random.randint(1, 41)
season = 1
```

```
if j >= 1:
season = 2
while judge(field[i + 26][5], field[i + 26][j + 1], plan_2024[i + 26][0], plan_2024
[i + 26][1],plan_2024[i + 26][2], plan_2024[i + 26][3], season) == False:
field[i + 26][j + 1] = random.randint(1, 41)

for i in range(16): # 普通大棚
for j in range(4):
season = 1
if j >= 2:
season = 2
if judge_bean(plan_2023[i + 26 + 8][0],plan_2023[i + 26 + 8][1]) or judge_bean(plan_
2024[i + 26 + 8][0],plan_2024[i + 26 + 8][1]):
field[i + 26 + 8][j] = random.randint(1, 41)
while judge(field[i + 26 + 8][5], field[i + 26 + 8][j], plan_2024[i + 26 + 8][0], p
lan_2024[i + 26 + 8][1],plan_2024[i + 26 + 8][2], plan_2024[i + 26 + 8][3], season) == False:
field[i + 26 + 8][j] = random.randint(1, 41)
else:
field[i + 26 + 8][j]=random.choice([17,18,19])

# field[i + 26 + 8][j] = random.randint(1, 41)
# while judge(field[i + 26 + 8][5], field[i + 26 + 8][j], plan_2023[i + 26 + 8][0],
 plan_2023[i + 26 + 8][1],plan_2023[i + 26 + 8][2], plan_2023[i + 26 + 8][3], season) == False:
#     field[i + 26 + 8][j] = random.randint(1, 41)

for i in range(4): # 智能大棚
for j in range(4):
season = 1
if j >= 2:
season = 2
if judge_bean(plan_2023[i + 26 + 8 + 16][0],plan_2023[i + 26 + 8 + 16][1]) or
 judge_bean(plan_2023[i + 26 + 8 + 16][2],plan_2023[i + 26 + 8 + 16][3]) or
  judge_bean(plan_2024[i + 26 + 8 + 16][0],plan_2024[i + 26 + 8 + 16][1]) or
  judge_bean(plan_2024[i + 26 + 8 + 16][2],plan_2024[i + 26 + 8 + 16][3]) :

field[i + 26 + 8 + 16][j] = random.randint(1, 41)
else:
field[i + 26 + 8 + 16][j] = random.choice([17, 18, 19])
# field[i + 26 + 8 + 16][j] = random.randint(1, 41)
if season == 1:
while judge(field[i + 26 + 8 + 16][5], field[i + 26 + 8 + 16][j], plan_2024[i + 26 +
 8 + 16][0],plan_2024[i + 26 + 8 + 16][1], plan_2024[i + 26 + 8 + 16][2], plan_2024
 [i + 26 + 8 + 16][3],season) == False:
field[i + 26 + 8 + 16][j] = random.randint(1, 41)
else:
while judge(field[i + 26 + 8 + 16][5], field[i + 26 + 8 + 16][j], plan_2024[i + 26 +
 8 + 16][0],plan_2024[i + 26 + 8 + 16][1], plan_2024[i + 26 + 8 + 16][2], plan_2024
```

```
 [i + 26 + 8 + 16][3],season) == False or judge(field[i + 26 + 8 + 16][5], field[i
 + 26 + 8 + 16][j],field[i + 26 + 8 + 16][0], field[i + 26 + 8 + 16][1],plan_2023[
 i + 26 + 8 + 16][2], plan_2023[i + 26 + 8 + 16][3],season) == False:
field[i + 26 + 8 + 16][j] = random.randint(1, 41)
return True

def cost(* field):
#print(cost_2023)
cost_sum=0
for i in range(54):
x=int(field[i][0])
y=int(field[i][5]-1)
if field[i][1]==0: # 单季产品
cost_sum+=cost_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]
else:
cost_sum+=cost_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*0.5+cost_2023
[int(field[i][1]-1)][int(field[i][5]-1)]*field[i][4]*0.5+cost_2023[int(field[i][2]-1)]
[int(field[i][5]-1)]*field[i][4]*0.5+cost_2023[int(field[i][3]-1)][int(field[i][5]-1)]*field[i][4]*0.5
return cost_sum

def profit_cal(year,target,* field):
profit_sum=0
if target==1:
for i in range(54):
x=int(field[i][0]-1)
y=int(field[i][5]-1)
if x<=16:
if x==6 or x==7:
if field[i][1] == 0:
profit_sum += min(yield_2023[int(field[i][0] - 1)][int(field[i][5] - 1)] * field[i]
[4] * 0.9 ** (year),yield_2023[int(field[i][0] - 1)][int(field[i][5] - 1)] * field[i]
[4] * 1.05**(year)) * (price_2023[int(field[i][0] - 1)][int(field[i][5] - 1)][0]
+price_2023[int(field[i][0] - 1)][int(field[i][5] - 1)][1]) / 2
else:
for k in range(4):
profit_sum += min(yield_2023[int(field[i][k] - 1)][int(field[i][5] - 1)] *
field[i][4] * 0.9 ** (year),yield_2023[int(field[i][k] - 1)][int(field[i][5] - 1)]
 * field[i][4] *1.05**(year)) * (price_2023[int(field[i][k] - 1)][int(field[i][5]
 - 1)][0] +price_2023[int(field[i][k] - 1)][int(field[i][5] - 1)][1]) / 2 * 0.5

else:
if field[i][1]==0:
profit_sum+=min(yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*
0.9**(year),yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*1.05)*
(price_2023[int(field[i][0]-1)][int(field[i][5]-1)][0]+price_2023[int(field[i][0]
-1)][int(field[i][5]-1)][1])/2
else:
```

```
for k in range(4):
profit_sum+=min(yield_2023[int(field[i][k]-1)][int(field[i][5]-1)]*field[i][4]*
 0.9**(year),yield_2023[int(field[i][k]-1)][int(field[i][5]-1)]*field[i][4]*1.05)*(
price_2023[int(field[i][k]-1)][int(field[i][5]-1)][0]+price_2023[int(field[i][k]-1)
][int(field[i][5]-1)][1])/2*0.5
elif x<=37:
if field[i][1]==0:
profit_sum+=min(yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]* 0.9**(year)
,yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*(1-0.0108
)*1.05)*(price_2023[int(field[i][0]-1)][int(field[i][5]-1)][0]+price_2023[int(field[i][0]-1)
][int(field[i][5]-1)][1])/2*1.05**(year)
else:
for k in range(4):
profit_sum+=min(yield_2023[int(field[i][k]-1)][int(field[i][5]-1)]*field[i][4]* 0.9**(year)
,yield_2023[int(field[i][k]-1)][int(field[i][5]-1)]*field[i][4]*(1-0.0108)*1.05)*
(price_2023[int(field[i][k]-1)][int(field[i][5]-1)][0]+price_2023[int(field[i][k]-1)][int
(field[i][5]-1)][1])/2*0.5*1.05**(year)
elif x<41:
if field[i][1] == 0:
profit_sum += min(yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]* 0.9**(year),
yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*(1-0.0108)*1.05)* (price_2023
[int(field[i][0] - 1)][int(field[i][5] - 1)][0] + price_2023[int(field[i][0] - 1)]
[int(field[i][5] - 1)][1]) / 2 * 0.99 ** ( year)
else:
for k in range(4):
profit_sum +=min(yield_2023[int(field[i][k]-1)][int(field[i][5]-1)]*field[i][4]* 0.9**(year
),yield_2023[int(field[i][k]-1)][int(field[i][5]-1)]*field[i][4]*(1-0.0108)*1.05)*
(price_2023[int(field[i][k] - 1)][int(field[i][5] - 1)][0] +price_2023[int(field[i][k] - 1)]
[int(field[i][5] - 1)][1]) / 2 * 0.5 *0.99 ** (year)
else:
if field[i][1] == 0:
profit_sum +=min(yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]* 0.9**(year)
,yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*(1-0.0108)*1.05)* (price_2023
[int(field[i][0] - 1)][int(field[i][5] - 1)][0] + price_2023[int(field[i][0] - 1)]
[int(field[i][5] - 1)][1]) / 2 * 0.95 ** ( year)
else:
for k in range(4):
profit_sum +=min(yield_2023[int(field[i][k]-1)][int(field[i][5]-1)]*field[i][4]* 0.9**(year)
,yield_2023[int(field[i][k]-1)][int(field[i][5]-1)]*field[i][4]*(1-0.0108)*1.05)* (price_2023
[int(field[i][k] - 1)][int(field[i][5] - 1)][0] +price_2023[int(field[i][k] - 1)][int(field[i
][5] - 1)][1]) / 2 * 0.5 *0.95 ** (year)

return profit_sum

#if sale_2023[int(field[i][0]-1)][int(field[i][5]-1)]<yield_2023[int(field[i][0]-1)][int(field
[i][5]-1)]*field[i][4]:
#profit_sum+=sale_2023[int(field[i][0]-1)][int(field[i][5]-1)]*random.uniform(price_2023[in
```

```
t(field[i][0]-1)][int(field[i][5]-1)][0],price_2023[int(field[i][0]-1)][int(field[i][5]-1)][1])
#else:
#profit_sum+=yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*random.uniform
(price_2023[int(field[i][0]-1)][int(field[i][5]-1)][0],price_2023[int(field[i][0]-1)][int(field[i][5]-1)][1])

else:
for i in range(54):
x=int(field[i][0]-1)
y=int(field[i][5]-1)
if x<=16:
if field[i][1]==0:
profit_sum+=yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*(price_2023[int
(field[i][0]-1)][int(field[i][5]-1)][0]+price_2023[int(field[i][0]-1)][int(field[i][5]-1)][1])/2
else:
for k in range(4):
profit_sum+=yield_2023[int(field[i][k]-1)][int(field[i][5]-1)]*field[i][4]*(price_2023
[int(field[i][k]-1)][int(field[i][5]-1)][0]+price_2023[int(field[i][k]-1)][int(field[i][5]-1)][1])/2*0.5
else:
if field[i][1]==0:
profit_sum+=(yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*(1-0.0108)+
yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*0.0108*0.5)*(price_2023
[int(field[i][0]-1)][int(field[i][5]-1)][0]+price_2023[int(field[i][0]-1)][int(field[i][5]-1)][1])/2
else:
for k in range(4):
profit_sum+=(yield_2023[int(field[i][k]-1)][int(field[i][5]-1)]*field[i][4]*(1-0.0108)+
yield_2023[int(field[i][k]-1)][int(field[i][5]-1)]*field[i][4]*0.0108*0.5)*(price_2023
[int(field[i][k]-1)][int(field[i][5]-1)][0]+price_2023[int(field[i][k]-1)][int(field[i][5]-1)][1])/2*0.5
return profit_sum

#if sale_2023[int(field[i][0]-1)][int(field[i][5]-1)]<yield_2023[int(field[i][0]-1)]
[int(field[i][5]-1)]*field[i][4]:
#profit_sum+=sale_2023[int(field[i][0]-1)][int(field[i][5]-1)]*random.uniform(price_
2023[int(field[i][0]-1)][int(field[i][5]-1)][0],price_2023[int(field[i][0]-1)][int
(field[i][5]-1)][1])+(yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i]
[4]-sale_2023[int(field[i][0]-1)][int(field[i][5]-1)])*0.5*random.uniform(price_20
23[int(field[i][0]-1)][int(field[i][5]-1)][0],price_2023[int(field[i][0]-1)][in
t(field[i][5]-1)][1])
#else:
#profit_sum+=yield_2023[int(field[i][0]-1)][int(field[i][5]-1)]*field[i][4]*random.
uniform(price_2023[int(field[i][0]-1)][int(field[i][5]-1)][0],price_2023[int(field
[i][0]-1)][int(field[i][5]-1)][1])



def target_1(year,*field):
result=profit_cal(year,1,*field)-cost(*field)*1.05**(year)
return result
```

```
def target_2(year,*field):
result = profit_cal(year,2,*field) - cost(*field)*1.05**(year)
return result

def judge(field_input,crop_input,crop_pre_1,crop_pre_2,crop_pre_3,crop_pre_4,crop_season):
 #判断该地块是否可以种植该作物
enable_123=[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]
enable_4_1=[17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34]
enable_4_2=[35,36,37]
enable_4_special=16
enable_5_1=[17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34]
enable_5_2=[38,39,40,41]
enable_6=[17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34]
#判断平旱地、梯田、山坡地种植情况
if field_input in [1,2,3]:
if crop_input in enable_123 and crop_input != crop_pre_1:
return True
else:
return False
elif field_input == 4:
if crop_season==0 and crop_input==enable_4_special:
if crop_input != crop_pre_1:
return True
else:
return False
else:
if crop_season==1 or crop_season==0:
if crop_input in enable_4_1 and ((crop_input !=crop_pre_1 and crop_input !=crop_pre_2)
 and (crop_input !=crop_pre_3 and crop_input !=crop_pre_4)):
return True
else:
return False
else:
if crop_input in enable_4_2 and ((crop_input !=crop_pre_1 and crop_input !=crop_pre_2)

else:
return False
elif field_input == 5:
if crop_season == 1:
if crop_input in enable_5_1 and ((crop_input != crop_pre_1 and crop_input != crop_pre_2)
 and (crop_input != crop_pre_3 and crop_input != crop_pre_4)):
return True
else:
return False
else:
```

```
if crop_input in enable_5_2 and ((crop_input != crop_pre_1 and crop_input != crop_pre_2)
and (crop_input != crop_pre_3 and crop_input != crop_pre_4)):
return True
else:
return False
elif field_input == 6:
    #注意使用field_input==6时，crop_pre_1和crop_pre_2需要替换为上一季度的值再进行一次judge
if crop_season == 1:
if crop_input in enable_6 and ((crop_input != crop_pre_1 and crop_input != crop_pre_2)
and (crop_input != crop_pre_3 and crop_input != crop_pre_4)):
return True
else:
return False
else:
if crop_input in enable_6 and ((crop_input != crop_pre_1 and crop_input != crop_pre_2)
 and (crop_input != crop_pre_3 and crop_input != crop_pre_4)):
return True
else:
return False
else:
return False


def judge_bean(crop_pre_1,crop_pre_2):
bean_type=[1,2,3,4,5,17,18,19]
if (crop_pre_1 not in bean_type) and (crop_pre_2 not in bean_type):
return False
else:
return True


#创建地块矩阵 field
#简化为平旱地、梯田、山坡地仅种植一种，水浇地、普通大棚、智慧大棚最多平分种植两种作物
file1=pd.read_excel(r"D:\STUDY\数模国赛2024\C题\附件1.xlsx",sheet_name="乡村的现有耕地")
#print(file1)
square=file1["地块面积/亩"]
field_type=file1["地块类型"]
#print(field_type)
field=np.zeros((54,6))
    #第一季度作物上半（单季），第一季度作物下半，第二季度作物上半，第二季度作物下半
，地块面积，地块类型 1.平旱地 2.梯田 3.山坡地 4.水浇地 5.普通大棚 6.智慧大棚
field1=np.zeros((54,6))
field2=np.zeros((54,6))
field3=np.zeros((54,6))
field4=np.zeros((54,6))
field5=np.zeros((54,6))
fBest=np.zeros((54,6)) #用于存储最优解
```

```
init_field(*field)
init_field(*field1)
init_field(*field2)
init_field(*field3)
init_field(*field4)
init_field(*field5)

#print(field)
#print(field1)

#创建作物矩阵 crop
file2=pd.read_excel(r"D:\STUDY\数模国赛2024\C题\附件1.xlsx",sheet_name="乡村种植的农作物")
crop=np.zeros((41,2)) # 作物编号,作物类型 1.粮食（豆类） 2.粮食 3.蔬菜（豆类） 4.蔬菜 5.食用菌

crop_number=file2["作物编号"]
#print(crop_number)
crop_kind=file2["作物类型"]
for i in range(41):
crop[i][0]=crop_number[i]
for i in range(41):
if crop_kind[i] == "粮食（豆类）":
crop[i][1]=1
elif crop_kind[i] == "粮食":
crop[i][1]=2
elif crop_kind[i] == "蔬菜（豆类）":
crop[i][1]=3
elif crop_kind[i] == "蔬菜":
crop[i][1]=4
else:
crop[i][1]=5
#print(crop)
#result1_1
# plan_2029=[[ 1,0,0,0],[ 4, 0, 0, 0],[ 7, 0, 0, 0],[ 3, 0, 0, 0],[11, 0, 0, 0],[ 5, 0, 0, 0],
#           [ 2, 0, 0, 0],[ 8, 0, 0, 0],[ 3, 0, 0, 0],[14, 0, 0, 0],[ 3, 0, 0, 0],[ 1, 0, 0, 0],
#           [13, 0, 0, 0],[ 3, 0, 0, 0],[ 8, 0, 0, 0],[ 2, 0, 0, 0],[ 3, 0, 0, 0],[ 4, 0, 0, 0],
#           [ 4, 0, 0, 0],[ 7, 0, 0, 0],[ 5, 0, 0, 0],[ 6, 0, 0, 0],[13, 0, 0, 0],[12, 0, 0, 0],
#           [ 2, 0, 0, 0],[12, 0, 0, 0],[17,31,37,37],[17,23,37,37],[17,29,35,37],[29,32,35,35],
#           [23,32,36,35],[28,23,36,37],[34,23,36,37],[17,18,35,36],[29,34,41,40],[19,18,17,19],
#           [18,19,19,17],[18,18,17,19],[18,17,18,18],[19,17,17,18],[32,21,38,38],[18,19,19,19],
#           [18,17,17,19],[17,18,17,17],[22,26,38,40],[30,32,38,41],[18,17,17,17],[21,20,38,41],
#           [34,33,38,38],[18,27,38,38],[27,21,19,33],[17,17,19,27],[32,32,21,31],[34,31,33,28]]
#
#
# plan_2028=[[14,0,0,0],[10, 0, 0, 0],[ 4, 0, 0, 0],[12, 0, 0, 0],[14, 0, 0, 0],[ 9, 0, 0, 0],
#           [ 4, 0, 0, 0],[ 3, 0, 0, 0],[ 1, 0, 0, 0],[ 9, 0, 0, 0],[15, 0, 0, 0],[13, 0, 0, 0],
#           [ 6, 0, 0, 0],[12, 0, 0, 0],[14, 0, 0, 0],[12, 0, 0, 0],[ 6, 0, 0, 0],[13, 0, 0, 0],
```

```
#          [ 2, 0, 0, 0],[ 1, 0, 0, 0],[12, 0, 0, 0],[ 8, 0, 0, 0],[ 1, 0, 0, 0],[ 2, 0, 0, 0],
#          [12, 0, 0, 0],[ 2, 0, 0, 0],[18,29,36,36],[32,21,35,35],[19,19,36,36],[34,23,37,37],
#          [19,24,37,37],[18,20,35,35],[17,22,35,35],[32,23,23,34],[33,31,39,38],[25,25,41,41],
#          [23,23,41,40],[30,33,41,41],[27,32,41,41],[20,29,31,19],[23,23,40,40],[27,32,25,17],
#          [25,24,39,40],[23,22,38,41],[18,25,27,30],[24,31,40,40],[24,25,39,40],[28,22,22,24],
#          [28,27,40,40],[17,21,41,40],[30,26,22,17],[23,23,30,21],[23,17,34,33],[23,18,29,32]]
#
#
# plan_2027=[[ 9, 0, 0, 0],[13, 0, 0, 0],[ 7, 0, 0, 0],[11, 0, 0, 0],[ 2, 0, 0, 0],[14, 0, 0,
    0],
#          [12, 0, 0, 0],[ 2, 0, 0, 0],[ 4, 0, 0, 0],[ 3, 0, 0, 0],[ 9, 0, 0, 0],[ 9, 0, 0, 0],
#          [ 3, 0, 0, 0],[ 4, 0, 0, 0],[ 2, 0, 0, 0],[10, 0, 0, 0],[ 4, 0, 0, 0],[ 8, 0, 0, 0],
#          [ 8, 0, 0, 0],[12, 0, 0, 0],[15, 0, 0, 0],[ 4, 0, 0, 0],[ 6, 0, 0, 0],[10, 0, 0, 0],
#          [ 9, 0, 0, 0],[10, 0, 0, 0],[22,25,35,37],[30,20,37,36],[17,22,37,35],[21,17,36,35],
#          [34,25,36,35],[32,23,37,37],[19,28,37,36],[27,24,23,29],[17,18,19,17],[34,29,39,38],
#          [20,28,39,39],[25,34,39,40],[24,25,39,38],[34,25,38,41],[18,19,19,18],[23,25,39,39],
#          [28,32,41,38],[28,27,39,40],[23,34,40,38],[17,17,19,19],[26,21,38,38],[18,19,17,17],
#          [18,31,38,39],[19,23,39,38],[19,32,33,23],[26,27,30,28],[18,30,20,25],[20,25,27,24]]
#
# plan_2026=[[ 6,0,0,0],[ 3,0,0,0],[13,0,0,0],[ 2,0,0,0],[11,0,0,0],[ 2,0,0,0],
#          [ 6,0,0,0],[ 1,0,0,0],[11,0,0,0],[ 2,0,0,0],[ 1,0,0,0],[ 3,0,0,0],
#          [15,0,0,0],[11,0,0,0],[13,0,0,0],[ 4,0,0,0],[ 3,0,0,0],[ 2,0,0,0],
#          [ 7,0,0,0],[ 9,0,0,0],[ 5,0,0,0],[12,0,0,0],[12,0,0,0],[ 9,0,0,0],
#          [ 4,0,0,0],[15,0,0,0],[31,8,5,7],[27,4,6,6],[32,9,7,7],[31,0,7,7],
#          [17,3,5,7],[17,9,7,6],[29,0,5,6],[18,5,6,6],[33,9,7,8],[19,8,0,9],
#          [24,4,9,2],[18,8,8,8],[17,5,9,1],[23,9,8,0],[27,8,0,3],[22,3,4,2],
#          [18,7,7,9],[17,7,9,9],[24,0,0,8],[25,7,9,1],[19,8,9,8],[32,0,9,9],
#          [19,7,8,9],[18,9,9,7],[17,7,8,2],[18,9,7,3],[29,8,0,5],[30,7,1,4]]
#
# plan_2025=[[5,0,0,0],[10, 0, 0, 0],[ 5, 0, 0, 0],[ 8, 0, 0, 0],[15, 0, 0, 0],[11, 0, 0, 0],
#          [ 1, 0, 0, 0],[12, 0, 0, 0],[ 7, 0, 0, 0],[ 8, 0, 0, 0],[ 7, 0, 0, 0],[11, 0, 0, 0],
#          [ 2, 0, 0, 0],[ 5, 0, 0, 0],[ 6, 0, 0, 0],[10, 0, 0, 0],[ 5, 0, 0, 0],[14, 0, 0, 0],
#          [ 4, 0, 0, 0],[ 1, 0, 0, 0],[15, 0, 0, 0],[10, 0, 0, 0],[ 2, 0, 0, 0],[ 2, 0, 0, 0],
#          [ 7, 0, 0, 0],[ 3, 0, 0, 0],[29,19,36,36],[33,19,37,35],[16,21,35,36],[19,32,35,35],
#          [27,26,36,36],[24,29,35,35],[27,23,37,37],[32,21,35,37],[26,32,41,41],[21,21,41,38],
#        [17,27,41,38],[21,32,39,38],[23,27,40,38],[18,34,39,41],[27,23,40,38],[27,29,40,39],
#        [32,21,41,41],[21,18,38,41],[18,30,41,41],[22,34,40,40],[24,21,39,39],[24,23,41,41],
#        [20,28,39,39],[32,29,39,41],[19,22,34,29],[25,26,22,22],[24,31,21,21],[27,27,26,25]]
#
#
# plan_2024=[[2,0,0,0],[15, 0, 0, 0],[ 8, 0, 0, 0],[ 9, 0, 0, 0],[10, 0, 0, 0],[14, 0, 0, 0],
#          [ 3, 0, 0, 0],[ 6, 0, 0, 0],[ 2, 0, 0, 0],[13, 0, 0, 0],[ 8, 0, 0, 0],[ 1, 0, 0, 0],
#          [ 1, 0, 0, 0],[ 2, 0, 0, 0],[ 2, 0, 0, 0],[ 6, 0, 0, 0],[ 4, 0, 0, 0],[11, 0, 0, 0],
#          [ 3, 0, 0, 0],[ 6, 0, 0, 0],[ 6, 0, 0, 0],[ 3, 0, 0, 0],[ 4, 0, 0, 0],[ 7, 0, 0, 0],
#          [12, 0, 0, 0],[12, 0, 0, 0],[18,22,37,35],[25,23,36,36],[23,19,37,37],[29,26,37,36],
#          [25,21,37,35],[26,34,36,36],[30,17,36,36],[28,29,36,36],[33,31,39,40],[22,17,39,40],
#          [18,28,39,40],[22,30,41,40],[34,18,41,41],[25,31,40,38],[22,24,39,41],[34,24,38,41],
```

```
#          [23,27,39,39],[27,23,40,39],[33,22,38,40],[19,33,38,39],[28,34,38,40],[17,31,40,38],
#          [29,32,38,38],[28,27,38,38],[27,30,31,17],[33,23,34,30],[33,19,18,29],[29,31,18,20]]

#2023的种植情况，按照field土地顺序

#result1_2

#result 2
plan_2029=[[10,0,0,0],[ 2, 0, 0, 0],[ 8, 0, 0, 0],[ 6, 0, 0, 0],[ 9, 0, 0, 0],[ 2, 0, 0, 0],
[11, 0, 0, 0],[ 7, 0, 0, 0],[ 5, 0, 0, 0],[ 5, 0, 0, 0],[ 6, 0, 0, 0],[ 9, 0, 0, 0],[ 4, 0, 0,
    0],
[ 3, 0, 0, 0],[ 9, 0, 0, 0],[13, 0, 0, 0],[ 2, 0, 0, 0],[11, 0, 0, 0],[ 5, 0, 0, 0]
,[14, 0, 0, 0],[ 1, 0, 0, 0],[ 4, 0, 0, 0],[ 1, 0, 0, 0],[ 5, 0, 0, 0],[ 2, 0, 0, 0],[ 2, 0,
    0,
    0],[24,32,36,36],[28,33,35,35],[18,29,37,37],[23,25,37,37],[22,17,36,36],[29,22,36,36],[18,17,36,36]
,[29,29,36,36],[17,18,19,17],[22,32,18,27],[17,19,17,17],[18,24,40,39],[29,28,41,39],[23,21,41,39],
[26,21,39,38],[19,17,17,17],[17,17,19,19],[32,25,41,39],[19,17,19,19],[17,19,18,18],[28,31,40,41]
,[19,33,41,41],[32,23,40,40],[26,32,39,39],[17,19,33,21],[27,31,29,21],[31,27,25,20]
,[30,20,26,34]]

plan_2028=[[ 3,0,0,0],[12, 0, 0, 0],[ 2, 0, 0, 0],[13, 0, 0, 0],[12, 0, 0, 0],[13, 0, 0, 0],
[ 2, 0, 0, 0],[ 5, 0, 0, 0],[12, 0, 0, 0],[14, 0, 0, 0],[ 1, 0, 0, 0],[13, 0, 0, 0],[ 3, 0, 0,
    0]
,[14, 0, 0, 0],[ 4, 0, 0, 0],[ 4, 0, 0, 0],[13, 0, 0, 0],[ 3, 0, 0, 0],[ 8, 0, 0, 0],
[ 5, 0, 0, 0],[ 6, 0, 0, 0],[ 2, 0, 0, 0],[ 8, 0, 0, 0],[12, 0, 0, 0],[12, 0, 0, 0],
[11, 0, 0, 0],[19,17, 35,35],[17,23, 37,36],[24,26, 27,28],[18,28, 36,35],[33,33, 37,35]
,[17,33, 37,37],[29,24, 35,37],[17,28, 37,35],[22,21, 32,32],[23,18, 28,34],[28,27, 17,18],
[25,21, 33,32],[18,18, 19,17],[19,19, 17,18],[17,19, 19,17],[33,23, 40,40],[23,34, 34,26],
[19,17, 18,19],[24,33, 40,40],[33,25, 34,31],[33,33, 38,38],[22,28, 38,40],[18,19, 17,18],
[19,19, 18,18],[27,32, 22,22],[34,18, 22,33],[18,19, 17,24],[32,27, 24,17]]


plan_2027=[[13,0,0,0],[10, 0, 0, 0],[ 5, 0, 0, 0],[ 2, 0, 0, 0],[ 5, 0, 0, 0],[ 2, 0, 0, 0]
,[ 9, 0, 0, 0],[10, 0, 0, 0],[ 7, 0, 0, 0],[ 6, 0, 0, 0],[12, 0, 0, 0],[ 3, 0, 0, 0],[15, 0,
    0, 0]
,[ 5, 0, 0, 0],[ 8, 0, 0, 0],[ 7, 0, 0, 0],[ 9, 0, 0, 0],[ 9, 0, 0, 0],[15, 0, 0, 0],[10, 0,
    0, 0]
[ 4, 0, 0, 0],[10, 0, 0, 0],[ 7, 0, 0, 0],[ 4, 0, 0, 0],[12, 0, 0, 0],[ 9, 0, 0,
    0],[32,34,36,36],
[31,31,35,35],[29,23,36,36],[29,34,37,37],[29,18,27,21],[26,26,32,19],[32,30,36,36]
,[23,34,36,36],[25,27,40,38],[18,34,39,38],[33,24,40,41],[19,33,39,38],[31,29,34,22],[29,29,21,23]
,[33,33,25,30],[25,29,41,38],[20,24,39,39],[21,24,28,26],[32,21,38,39],[20,30,24,29],[19,17,19,18]
,[19,17,17,18],[32,29,41,41],[32,31,39,40],[29,30,23,20],[29,17,24,19],[34,23,23,31],[26,28,31,33]]

plan_2026=[[ 9,0,0,0],[13, 0, 0, 0],[ 7, 0, 0, 0],[11, 0, 0, 0],[ 7, 0, 0, 0],[10, 0, 0, 0],
[10, 0, 0, 0],[ 4, 0, 0, 0],[ 5, 0, 0, 0],[ 3, 0, 0, 0],[ 7, 0, 0, 0],[11, 0, 0, 0],[ 7, 0, 0,
    0]
```

```
,[ 9, 0, 0, 0],[14, 0, 0, 0],[ 2, 0, 0, 0],[ 3, 0, 0, 0],[ 4, 0, 0, 0],[ 3, 0, 0, 0],[ 9, 0,
    0, 0],
[14, 0, 0, 0],[13, 0, 0, 0],[ 2, 0, 0, 0],[13, 0, 0, 0],[12, 0, 0, 0],[ 4, 0, 0,
    0],[29,31,37,35],
[32,22,37,36],[31,31,30,33],[28,32,35,35],[32,29,37,36],[16,33,36,35]
,[29,19,35,37],[27,29,25,19],[18,17,19,17],[26,30,22,31],[20,29,28,34],[17,28,40,40],[24,28,38,40]
,[34,31,39,39],[31,29,39,40],[19,17,18,17],[21,17,41,40],[25,27,24,23],[19,18,17,19],[21,17,38,40]
,[25,26,40,39],[34,27,24,18],[24,23,40,40],[26,30,41,38],[18,22,17,18],[26,30,23,20],[24,24,23,25],[17,17,25,29

plan_2025=[[ 4,0,0,0],[ 2, 0, 0, 0],[12, 0, 0, 0],[ 7, 0, 0, 0],[11, 0, 0, 0],[ 3, 0, 0, 0]
,[ 4, 0, 0, 0],[ 6, 0, 0, 0],[12, 0, 0, 0],[ 7, 0, 0, 0],[ 1, 0, 0, 0],[ 7, 0, 0, 0],[ 3, 0,
    0, 0],
[14, 0, 0, 0],[ 4, 0, 0, 0],[ 1, 0, 0, 0],[ 9, 0, 0, 0],[ 1, 0, 0, 0],[ 5, 0, 0, 0],[ 4, 0, 0,
    0],
[11, 0, 0, 0],[ 2, 0, 0, 0],[10, 0, 0, 0],[ 1, 0, 0, 0],[ 3, 0, 0, 0],[14, 0, 0,
    0],[17,33,36,36],
[19,21,35,35],[17,17,35,35],[18,18,37,37],[19,17,35,35],[17,32,37,37],[18,34,36,36]
,[19,31,35,35],[21,34,38,40],[17,18,18,19],[17,19,17,18],[20,20,41,41],[17,18,18,19],[25,26,40,40]
,[18,26,38,38],[32,32,38,38],[34,19,39,38],[34,31,41,41],[33,32,41,40],[24,28,41,41],[32,33,41,38],
[33,32,39,39],[19,19,17,18],[19,19,17,17],[19,22,27,25],[18,19,25,28],[22,27,19,19],[32,28,20,27]]

plan_2024=[[14,0,0,0],[12, 0, 0, 0],[ 2, 0, 0, 0],[ 8, 0, 0, 0],[ 5, 0, 0, 0],[10, 0, 0, 0],
[15, 0, 0, 0],[ 9, 0, 0, 0],[13, 0, 0, 0],[ 9, 0, 0, 0],[15, 0, 0, 0],[ 3, 0, 0, 0],
[12, 0, 0, 0],[ 4, 0, 0, 0],[ 8, 0, 0, 0],[ 9, 0, 0, 0],[14, 0, 0, 0],[12, 0, 0, 0],
[ 8, 0, 0, 0],[ 2, 0, 0, 0],[ 2, 0, 0, 0],[11, 0, 0, 0],[ 7, 0, 0, 0],[ 4, 0, 0, 0],
[10, 0, 0, 0],[ 8, 0, 0, 0],[27,34,37,37],[16,24,36,37],[32,27,36,36],[21,23,36,36],
[24,32,37,37],[24,33,35,36],[19,21,37,35],[26,25,36,37],[25,28,41,39],[32,34,39,41],
[34,32,39,41],[34,17,40,38],[23,30,38,40],[20,17,38,41],[24,31,39,41],[26,22,41,39],
[25,21,41,41],[19,23,38,38],[34,21,39,38],[26,17,38,39],[23,19,39,40],[18,22,38,40],
[26,29,38,40],[22,27,40,40],[17,23,18,28],[20,30,21,27],[31,24,34,33],[22,21,18,31]]


plan_2023=[[6,0,0,0],[7,0,0,0],[7,0,0,0],[1,0,0,0],[4,0,0,0],[8,0,0,0],
[6,0,0,0],[2,0,0,0],[3,0,0,0],[4,0,0,0],[5,0,0,0],[8,0,0,0],
[6,0,0,0],[8,0,0,0],[9,0,0,0],[10,0,0,0],[1,0,0,0],[7,0,0,0],
[14,0,0,0],[15,0,0,0],[11,0,0,0],[12,0,0,0],[1,0,0,0],[13,0,0,0],
[6,0,0,0],[3,0,0,0],[20,20,36,36],[28,28,35,35],[21,21,35,35],[22,22,35,35],
[17,17,36,36],[18,18,37,37],[16,0,0,0],[16,0,0,0],[18,18,38,38],[24,24,38,38],
[25,25,38,38],[26,26,39,39],[28,28,39,39],[27,27,39,39],[19,19,40,40],[19,19,40,40],
[18,18,40,40],[17,17,41,41],[17,17,41,41],[22,22,41,41],[21,21,41,41],[29,29,41,41],
[30,27,41,41],[31,31,41,41],[32,33,24,21],[25,26,22,29],[17,17,28,30],[19,19,34,23]]

#2023亩产量，产品编号*土地类型
yield_2023=[[400,380,360,0,0,0],[500,475,450,0,0,0],[400,380,360,0,0,0],[350,330,315,0,0,0],[415,395,375,0,0,0]
,[100,950,900,0,0,0],[400,380,360,0,0,0],[630,600,570,0,0,0],[525,500,475,0,0,0],[110,105,100,0,0,0],[3000,2850
,[2200,2100,2000,0,0,0],[420,400,380,0,0,0],[525,500,475,0,0,0],[0,0,0,500,0,0],[0,0,0,3000,3600,3200],[0,0,0,2
,[0,0,0,3000,3600,3200],[0,0,0,2000,2400,2200],[0,0,0,2400,3000,2700],[0,0,0,6400,8000,7200],[0,0,0,2700,3300,3
```

```
,[0,0,0,3300,4000,3600],[0,0,0,3700,4500,4100],[0,0,0,4100,5000,4500],[0,0,0,3200,4000,3600],[0,0,0,12000,15000
,[0,0,0,1600,2000,1800],[0,0,0,10000,12000,11000],[0,0,0,5000,6000,5400],[0,0,0,5500,6600,6000],[0,0,0,5000,0,0
,[0,0,0,3000,0,0],[0,0,0,0,5000,0],[0,0,0,0,4000,0],[0,0,0,0,10000,0],[0,0,0,0,1000,0]]

#2023种植成本，产品编号*土地类型
cost_2023=[[400,400,400,0,0,0],[400,400,400,0,0,0],[350,350,350,0,0,0],[350,350,350,0,0,0],[350,350,350,0,0,0],
[500,500,500,0,0,0],[360,360,360,0,0,0],[400,400,400,0,0,0],[360,360,360,0,0,0],[350,350,350,0,0,0],[100,1000,1
[200,2000,2000,0,0,0],[400,400,400,0,0,0],[350,350,350,0,0,0],[0,0,0,680,0,0],[0,0,0,2000,2400,2640],[0,0,0,100
[0,0,0,2000,2400,2640],[0,0,0,2000,2400,2640],[0,0,0,2000,2400,2640],[0,0,0,2000,2400,2640],[0,0,0,2300,2700,30
[0,0,0,2400,3000,3300],[0,0,0,2900,3500,3850],[0,0,0,1600,2000,2200],[0,0,0,1600,2000,2200],[0,0,0,2900,3500,38
[0,0,0,1000,1200,1300],[0,0,0,4100,5000,5500],[0,0,0,2000,2500,2750],[0,0,0,900,1100,1200],[0,0,0,2000,0,0],[0,
[0,0,0,0,3000,0],[0,0,0,0,2000,0],[0,0,0,0,10000,0],[0,0,0,0,10000,0]]

#2023种植销售单价，产品编号*土地类型
price_2023=[[[2.50,4.00],[2.50,4.00],[2.50,4.00],[0,0],[0,0],[0,0]],
[[6.50,8.50],[6.50,8.50],[6.50,8.50],[0,0],[0,0],[0,0]],
[[7.50,9.00],[7.50,9.00],[7.50,9.00],[0,0],[0,0],[0,0]],
[[6.00,8.00],[6.00,8.00],[6.00,8.00],[0,0],[0,0],[0,0]],
[[6.00,7.50],[6.00,7.50],[6.00,7.50],[0,0],[0,0],[0,0]],
[[3.00,4.00],[3.00,4.00],[3.00,4.00],[0,0],[0,0],[0,0]],
[[2.50,3.50],[2.50,3.50],[2.50,3.50],[0,0],[0,0],[0,0]],
[[6.00,7.50],[6.00,7.50],[6.00,7.50],[0,0],[0,0],[0,0]],
[[5.50,6.50],[5.50,6.50],[5.50,6.50],[0,0],[0,0],[0,0]],
[[6.50,8.50],[6.50,8.50],[6.50,8.50],[0,0],[0,0],[0,0]],
[[30.00,50.00],[30.00,50.00],[30.00,50.00],[0,0],[0,0],[0,0]],
[[1.00,2.00],[1.00,2.00],[1.00,2.00],[0,0],[0,0],[0,0]],
[[2.50,4.00],[2.50,4.00],[2.50,4.00],[0,0],[0,0],[0,0]],
[[5.00,6.00],[5.00,6.00],[5.00,6.00],[0,0],[0,0],[0,0]],
[[3.00,4.00],[3.00,4.00],[3.00,4.00],[0,0],[0,0],[0,0]],
[[0,0],[0,0],[0,0],[6.00,8.00],[0,0],[0,0]],
[[0,0],[0,0],[0,0],[7.00,9.00],[7.00,9.00],[8.40,10.80]],
[[0,0],[0,0],[0,0],[5.50,8.00],[5.50,8.00],[6.60,9.60]],
[[0,0],[0,0],[0,0],[5.00,8.00],[5.00,8.00],[6.00,9.60]],
[[0,0],[0,0],[0,0],[3.00,4.50],[3.00,4.50],[3.60,5.40]],
[[0,0],[0,0],[0,0],[5.00,7.50],[5.00,7.50],[6.00,9.00]],
[[0,0],[0,0],[0,0],[5.00,6.00],[5.00,6.00],[6.00,7.20]],
[[0,0],[0,0],[0,0],[4.80,6.70],[4.80,6.70],[5.80,8.00]],
[[0,0],[0,0],[0,0],[4.00,6.50],[4.00,6.50],[5.80,7.80]],
[[0,0],[0,0],[0,0],[5.00,6.00],[5.00,6.00],[6.00,7.20]],
[[0,0],[0,0],[0,0],[5.50,7.50],[5.50,7.50],[6.60,9.00]],
[[0,0],[0,0],[0,0],[4.00,6.00],[4.00,6.00],[4.80,7.20]],
[[0,0],[0,0],[0,0],[5.00,6.50],[5.00,6.50],[6.00,7.80]],
[[0,0],[0,0],[0,0],[6.00,8.00],[6.00,8.00],[7.20,9.60]],
[[0,0],[0,0],[0,0],[4.50,6.00],[4.50,6.00],[5.40,7.20]],
[[0,0],[0,0],[0,0],[6.00,8.50],[6.00,8.50],[7.20,10.20]],
[[0,0],[0,0],[0,0],[3.00,6.00],[3.00,6.00],[3.60,7.20]],
[[0,0],[0,0],[0,0],[4.00,5.00],[4.00,5.00],[4.80,6.00]],
```

```
[[0,0],[0,0],[0,0],[3.20,4.80],[3.20,4.80],[3.80,5.80]],
[[0,0],[0,0],[0,0],[2.00,3.00],[0,0],[0,0]],
[[0,0],[0,0],[0,0],[2.00,3.00],[0,0],[0,0]],
[[0,0],[0,0],[0,0],[2.50,4.00],[0,0],[0,0]],
[[0,0],[0,0],[0,0],[0,0],[50.00,65.00],[0,0]],
[[0,0],[0,0],[0,0],[0,0],[18.00,20.00],[0,0]],
[[0,0],[0,0],[0,0],[0,0],[14.00,18.00],[0,0]],
[[0,0],[0,0],[0,0],[0,0],[80.00,120.00],[0,0]]]

#遗传算法寻找最优解
#创建种群个体
creat(*field)
creat(*field1)
creat(*field2)
creat(*field3)
creat(*field4)
creat(*field5)
fitness=[0,0,0,0,0,0] #存储各个个体的适应度
Gtime=200#迭代次数
q_possibility=0.7 #发生交叉的概率
p_possibility=0.1 #发生变异的概率
fitBest=0 #记录最优适应度
for i in range(Gtime):
fitness[0] = target_1(7,*field)
fitness[1] = target_1(7,*field1)
fitness[2] = target_1(7,*field2)
fitness[3] = target_1(7,*field3)
fitness[4] = target_1(7,*field4)
fitness[5] = target_1(7,*field5)
maxfit=max(fitness)
minfit=min(fitness)
for j in range(len(fitness)):
if fitness[j]==maxfit and maxfit>fitBest:
fitBest=maxfit
findex=j #存储当前最优解所在位置
for j in range(len(fitness)):
if fitness[j]==minfit:
fDiscard=j #存储当前最劣解所在位置

if findex==0 and maxfit==fitBest:
fBest=field
elif findex==1 and maxfit==fitBest:
fBest=field1
elif findex==2 and maxfit==fitBest:
fBest=field2
elif findex==3 and maxfit==fitBest:
fBest=field3
```

```
elif findex==4 and maxfit==fitBest:
fBest=field4
elif findex == 5 and maxfit == fitBest:
fBest=field5

if fDiscard==0:#舍弃最劣的解，用新的种群个体替换
init_field(*field)
creat(*field)
elif fDiscard==1:
init_field(*field1)
creat(*field1)
elif fDiscard==2:
init_field(*field2)
creat(*field2)
elif fDiscard==3:
init_field(*field3)
creat(*field3)
elif fDiscard==4:
init_field(*field4)
creat(*field4)
else:
init_field(*field5)
creat(*field5)

#交叉操作
q=random.random()
if q<q_possibility:
x=random.randint(0,53)
for m in range(4):
temp=field[x][m]
field[x][m]=field1[x][m]
field1[x][m]=temp
q = random.random()
if q < q_possibility:
x = random.randint(0, 53)
for m in range(4):
temp = field1[x][m]
field1[x][m] = field2[x][m]
field2[x][m] = temp
q = random.random()
if q < q_possibility:
x = random.randint(0, 53)
for m in range(4):
temp = field2[x][m]
field2[x][m] = field3[x][m]
field3[x][m] = temp
q = random.random()
```

```
if q < q_possibility:
x = random.randint(0, 53)
for m in range(4):
temp = field3[x][m]
field3[x][m] = field4[x][m]
field4[x][m] = temp
q = random.random()
if q < q_possibility:
x = random.randint(0, 53)
for m in range(4):
temp = field4[x][m]
field4[x][m] = field5[x][m]
field5[x][m] = temp
#print(fitBest)

#变异操作
p = random.random()
if p < p_possibility:
x = random.randint(0, 53)
if x>26:
for m in range(4):
field[x][m]=random.randint(1,41)
while
    judge(field[x][5],field[x][m],plan_2023[x][0],plan_2023[x][1],plan_2023[x][2],plan_2023[x][3],1)==False:
field[x][m] = random.randint(1, 41)
else:
field[x][0] = random.randint(1, 41)
while judge(field[x][5], field[x][0], plan_2023[x][0], plan_2023[x][1], plan_2023[x][2],
    plan_2023[x][3],1) == False:
field[x][0] = random.randint(1, 41)
if p < p_possibility:
x = random.randint(0, 53)
if x>26:
for m in range(4):
field1[x][m]=random.randint(1,41)
while
    judge(field1[x][5],field1[x][m],plan_2023[x][0],plan_2023[x][1],plan_2023[x][2],plan_2023[x][3],1)==False:
field1[x][m] = random.randint(1, 41)
else:
field1[x][0] = random.randint(1, 41)
while judge(field1[x][5], field1[x][0], plan_2023[x][0], plan_2023[x][1], plan_2023[x][2],
    plan_2023[x][3],1) == False:
field1[x][0] = random.randint(1, 41)
if p < p_possibility:
x = random.randint(0, 53)
if x>26:
for m in range(4):
```

```
field2[x][m]=random.randint(1,41)
while
    judge(field2[x][5],field2[x][m],plan_2023[x][0],plan_2023[x][1],plan_2023[x][2],plan_2023[x][3],1)==False:
field2[x][m] = random.randint(1, 41)
else:
field2[x][0] = random.randint(1, 41)
while judge(field2[x][5], field2[x][0], plan_2023[x][0], plan_2023[x][1], plan_2023[x][2],
    plan_2023[x][3],1) == False:
field2[x][0] = random.randint(1, 41)
if p < p_possibility:
x = random.randint(0, 53)
if x>26:
for m in range(4):
field3[x][m]=random.randint(1,41)
while
    judge(field3[x][5],field3[x][m],plan_2023[x][0],plan_2023[x][1],plan_2023[x][2],plan_2023[x][3],1)==False:
field3[x][m] = random.randint(1, 41)
else:
field3[x][0] = random.randint(1, 41)
while judge(field3[x][5], field3[x][0], plan_2023[x][0], plan_2023[x][1], plan_2023[x][2],
    plan_2023[x][3],1) == False:
field3[x][0] = random.randint(1, 41)
if p < p_possibility:
x = random.randint(0, 53)
if x>26:
for m in range(4):
field4[x][m]=random.randint(1,41)
while
    judge(field4[x][5],field4[x][m],plan_2023[x][0],plan_2023[x][1],plan_2023[x][2],plan_2023[x][3],1)==False:
field4[x][m] = random.randint(1, 41)
else:
field4[x][0] = random.randint(1, 41)
while judge(field4[x][5], field4[x][0], plan_2023[x][0], plan_2023[x][1], plan_2023[x][2],
    plan_2023[x][3],1) == False:
field4[x][0] = random.randint(1, 41)
if p < p_possibility:
x = random.randint(0, 53)
if x>26:
for m in range(4):
field5[x][m]=random.randint(1,41)
while
    judge(field5[x][5],field5[x][m],plan_2023[x][0],plan_2023[x][1],plan_2023[x][2],plan_2023[x][3],1)==False:
field5[x][m] = random.randint(1, 41)
else:
field5[x][0] = random.randint(1, 41)
while judge(field5[x][5], field5[x][0], plan_2023[x][0], plan_2023[x][1], plan_2023[x][2],
    plan_2023[x][3],1) == False:
```

```
field5[x][0] = random.randint(1, 41)
print(fitBest)
print(fBest)
# a=cost(* field)
# print(a)
# a=profit_cal(* field)
# print(a)
# a=target_1(*field)
# print(a)
```

3.1 灾害正态分布检验

使用 SPSS 中夏皮洛-威尔克检验灾害损失率是否为正态分布。

正态性检验

	柯尔莫戈洛夫-斯米诺夫			夏皮洛-威尔克		
	统计	自由度	显著性	统计	自由度	显著性
灾害损失率	0.178	10	0.200	0.903	10	0.236

3.2 灾害正态分布检验描述

使用 SPSS 中的描述性统计对灾害损失率作以描述

描述

			统计	标准 错误
灾害损失率	平均值		0.96370	0.003887
	平均值的 95% 置信区间	下限	0.95491	
		上限	0.97249	
	5% 剪除后平均值		0.96361	
	中位数		0.96200	
	方差		0.000	
	标准 偏差		0.012293	
	最小值		0.948	
	最大值		0.981	
	全距		0.033	
	四分位距		0.025	
	偏度		0.330	0.687
	峰度		-1.308	1.334

附录 D 问题三相关代码和处理结果

4.1 问题三模型代码

第三问代码，根据拟合得到的函数计算的预期销售量等参数，对优化进行限制，求解最优种植方案

```
import numpy as np
import random

field=np.zeros((20,26))

square=[80,55,35,72,68,55,60,46,40,28,25,86,55,44,50,25,60,45,35,20,15,13,15,18,27,20]

yield_now=[400,500,400,350,415,800,1000,400,630,525,110,3000,2200,420,525]

cost=[420,420,367.5,367.5,367.5,472.5,525,378,420,378,367.5,1050,2100,420,367.5]

price=[3.25,7.5,8.25,7,6.75,
3.5,3,6.75,6,7.5,
40,1.5,3.25,5.5,3.5]

plan_2023=[6,7,7,1,4,8,6,2,3,4,5,8,6,8,9,10,1,7,14,15,11,12,1,13,6,3]

sale=[17841.41088,17841.41088,18833.86961,18833.86961,18833.86961,16966.00025,16182.91875,18624.49238,17841.4

#初始化种群
for j in range(20):
for i in range(26):
field[j][i]=random.randint(1,15)
while field[j][i]==plan_2023[i] and
    square[i]*yield_now[int(field[j][i])-1]>sale[int(field[j][i])-1]:
field[j][i] = random.randint(1, 15)

Gtime=200
q_poss=0.7
p_poss=0.1
fitness=[]
fitBest=0
fBest=[0,0,0,0,0,
0,0,0,0,0,
0,0,0,0,0,
0,0,0,0,0,
0,0,0,0,0,
0]
for i in range(20):
sum=0
for j in range(26):
sum+=square[j]*yield_now[int(field[i][j])-1]*0.9*price[int(field[i][j])-1]-square[j]*cost[int(field[i][j])-1]
fitness.append(sum)
for i in range(Gtime):
#选择最优个体
maxfit=max(fitness)
```

```
minfit=min(fitness)
for k in range(20):
if fitness[k]==maxfit:
maxindex=k
if fitness[k]==minfit:
minindex=k
if maxfit>fitBest:
fitBest=maxfit
for m in range(26):
fBest[m]=field[maxindex][m]

#轮盘赌法选择遗传至下一代的个体
fitness_sum=0
pan=[]
for q in range(20):
fitness_sum+=fitness[q]
fitness_chose=[0,0,0,0,0,
0,0,0,0,0,
0,0,0,0,0,
0,0,0,0,0]
fitness_cumsum = [0, 0, 0, 0, 0,
0, 0, 0, 0, 0,
0, 0, 0, 0, 0,
0, 0, 0, 0, 0]
for q in range(20):
fitness_chose[q]=fitness[q]/fitness_sum #计算适应度累积百分比
for q in range(20):
for m in range(q+1):
fitness_cumsum[q]+=fitness_chose[m]
for q in range(20):
pan.append(random.random())
pan.sort()
#print(pan)
for q in range(20):
if fitness_chose[q]<pan[q]:
for p in range(26):
field[q][p] = random.randint(1, 15)
while field[q][p] == plan_2023[p] and square[p] * yield_now[int(field[q][p]) - 1] >
    sale[int(field[q][p]) - 1]:
field[q][p] = random.randint(1, 15)
#模拟生物进化染色体交叉
for m in range (0,20,2):
for t in range(26):
q=random.random()
if q<q_poss:
temp=field[m][t]
field[m][t]=field[m+1][t]
```

```
field[m + 1][t]=temp
#模拟生物进化基因突变
for m in range(20):
for t in range(26):
p=random.random()
if p<p_poss:
field[m][t]=random.randint(1,15)
while field[m][t]==plan_2023[t]:
field[m][t] = random.randint(1, 15)
for x in range(20):
sum = 0
for y in range(26):
sum += square[y] * yield_now[int(field[x][y]) - 1] * price[int(field[x][y]) - 1] - square[y]
    * cost[int(field[x][y]) - 1] * price[int(field[x][y]) - 1]
fitness[x]=sum

print(fitBest)
print(fBest)
```

4.2 关系数矩阵热力图

相关系数矩阵热力图绘制代码

```
clc,clear
WeatherWeight=[1.0000 0.9197 -0.4950
0.9197   1.0000  -0.4007
-0.4950  -0.4007   1.0000];
LabelName={'种植成本','销售价格','预期销售量'};
hot_figure=heatmap(LabelName,LabelName,WeatherWeight)
mycolor=[127 169 205;63 96 163]./255;
colormap(gca,mycolor)
title('相关性系数矩阵');
```

4.3 拟合效果图

种植成本、销售价格和预期销售量的拟合效果图绘制代码

```
load nihe.mat

subplot(2,1,1)
y=sale(:,1);
x=cost(:,1);
z=price(:,1);
scatter(x,y,'filled','CData',[247 161 171]./255,'LineWidth',2,'MarkerFaceAlpha',0.5)
hold on
```

```
x1=linspace(1,10000,50000)
y1=-8070.4*log(x1)+77573
plot(x1,y1,'Color',[157 195 230]./255,'LineWidth',2)
xlabel('种植成本')
ylabel('预期销售量')
label={'实际值','拟合值'}
legend(label)
title('种植成本与预期销售量拟合效果图')
hold off
subplot(2,1,2)
scatter(z,y,'filled','CData',[157 195 230]./255,'LineWidth',2,'MarkerFaceAlpha',0.5)
hold on
x2=linspace(1,100,50000)
y2=-17703*log(x2)+52855
plot(x2,y2,'Color',[247 161 171]./255,'LineWidth',2)
xlabel('销售价格')
ylabel('预期销售量')
title('销售价格与预期销售量拟合效果图')
hold off
label={'实际值','拟合值'}
legend(label)
```

4.4 相关性前的数据展示图

第三问探究相关性前的数据展示图

```
load show.mat
x=linspace(1,62,62)
y1=show(:,1)
y2=show(:,2)
y3=show(:,3)
subplot(3,1,1)
bar(x,y1,'EdgeAlpha',0,'FaceColor',[251,180,174]./255,'FaceAlpha',0.7)
ylim([0,11000])
xticks('')
xlabel('种植成本')
legend('种植成本','Location','northwest')
title('销售指标数据对比展示图')
subplot(3,1,2)
bar(x,y2,'EdgeAlpha',0,'FaceColor',[179 205 227]./255,'FaceAlpha',0.7)
ylim([0,110])
xticks('')
xlabel('销售单价')
legend('销售单价','Location','northwest')
subplot(3,1,3)
bar(x,y3,'EdgeAlpha',0,'FaceColor',[204 235 197]./255,'FaceAlpha',0.7)
```

```
ylim([0,161000])
xticks('')
xlabel('预期销售量')
legend('预期销售量','Location','northwest')
```

4.5 拟合及其改进

直接拟合代码。

```
clear,clc
load Fit_Data.mat

%提取自变量
x1=Fit_Data(:,1);
x2=Fit_Data(:,2);
y=Fit_Data(:,3);

% 拟合: '销售单价与预期销售量'。
[xData, yData] = prepareCurveData( x2, y );

% 设置 fittype 和选项。
ft = fittype( 'log' );

% 对数据进行模型拟合。
[fitresult, gof] = fit( xData, yData, ft );

% 绘制数据拟合图。
figure( 'Name', '销售单价与预期销售量' );
h = plot( fitresult, xData, yData );
legend( h, 'y vs. x2', '销售单价与预期销售量', 'Location', 'NorthEast', 'Interpreter',
    'none' );
% 为坐标区加标签
xlabel( 'x2', 'Interpreter', 'none' );
ylabel( 'y', 'Interpreter', 'none' );
grid on

% 拟合: '种植成本与销售量'。
[xData, yData] = prepareCurveData( x1, y );

% 设置 fittype 和选项。
ft = fittype( 'log' );

% 对数据进行模型拟合。
[fitresult, gof] = fit( xData, yData, ft );

% 绘制数据拟合图。
```

```
figure( 'Name', '种植成本与销售量' );
h = plot( fitresult, xData, yData );
legend( h, 'y vs. x1', '种植成本与销售量', 'Location', 'NorthEast', 'Interpreter', 'none'
    );
% 为坐标区加标签
xlabel( 'x1', 'Interpreter', 'none' );
ylabel( 'y', 'Interpreter', 'none' );
grid on
```

拟合函数。

```
clear,clc

% 装载聚类分析后的两种类别数据
load Type_1.mat
load Type_2.mat

% 提取两类自变量
x11=Type_1(:,1);
x12=Type_1(:,2);
y1=Type_1(:,3);

x21=Type_2(:,1);
x22=Type_2(:,2);
y2=Type_2(:,3);

% 拟合: '类型一种植成本与销售量'。
[xData, yData] = prepareCurveData( x11, y1 );

% 设置 fittype 和选项。
ft = fittype( 'log' );
excludedPoints = excludedata( xData, yData, 'Indices', [18 32 33 34] );
opts = fitoptions( 'Method', 'LinearLeastSquares' );
opts.Exclude = excludedPoints;

% 对数据进行模型拟合。
[fitresult, gof] = fit( xData, yData, ft, opts );

% 绘制数据拟合图。
subplot(2,2,1)
%figure( 'Name', '类型一种植成本与销售量' );
plot( fitresult, xData, yData, excludedPoints );
legend('实际值', '离群值', '拟合值', 'Location', 'NorthEast', 'Interpreter', 'none' );
% 为坐标区加标签
xlabel( '', 'Interpreter', 'none' );
ylabel( '', 'Interpreter', 'none' );
xlim([1550,10000])
```

```
title('类型一种植成本与预期销售量','FontSize',6)
grid on

% 拟合: '类型一销售单价与销售量'。
[xData, yData] = prepareCurveData( x12, y1 );

% 设置 fittype 和选项。
ft = fittype( 'log' );
excludedPoints = excludedata( xData, yData, 'Indices', [19 20 33 34] );
opts = fitoptions( 'Method', 'LinearLeastSquares' );
opts.Exclude = excludedPoints;

% 对数据进行模型拟合。
[fitresult, gof] = fit( xData, yData, ft, opts );

% 绘制数据拟合图
subplot(2,2,2)
%figure( 'Name', '类型一销售单价与销售量' );
plot( fitresult, xData, yData, excludedPoints );
legend('实际值', '离群值', '拟合值', 'Location', 'NorthEast', 'Interpreter', 'none' );
% 为坐标区加标签
xlabel( '', 'Interpreter', 'none' );
ylabel( '', 'Interpreter', 'none' );
title('类型一销售单价与预期销售量','FontSize',6)
grid on

% 拟合: '类型二种植成本与销售量'。
[xData, yData] = prepareCurveData( x21, y2 );

% 设置 fittype 和选项。
ft = fittype( 'log' );
excludedPoints = excludedata( xData, yData, 'Indices', [1 4 11 22 24 27] );
opts = fitoptions( 'Method', 'LinearLeastSquares' );
opts.Exclude = excludedPoints;

% 对数据进行模型拟合。
[fitresult, gof] = fit( xData, yData, ft, opts );

% 绘制数据拟合图。
subplot(2,2,3)
%figure( 'Name', '类型二种植成本与销售量' );
plot( fitresult, xData, yData, excludedPoints );
legend('实际值', '离群值', '拟合值', 'Location', 'NorthEast', 'Interpreter', 'none');
% 为坐标区加标签
xlabel( '', 'Interpreter', 'none' );
ylabel( '', 'Interpreter', 'none' );
xlim([300,1200])
```

```
title('类型二种植成本与预期销售量','FontSize',6)
grid on

% 拟合: '类型二销售单价与销售量'。
[xData, yData] = prepareCurveData( x22, y2 );

% 设置 fittype 和选项。
ft = fittype( 'log' );
excludedPoints = excludedata( xData, yData, 'Indices', 21 );
opts = fitoptions( 'Method', 'LinearLeastSquares' );
opts.Exclude = excludedPoints;

% 对数据进行模型拟合。
[fitresult, gof] = fit( xData, yData, ft, opts );

% 绘制数据拟合图。
subplot(2,2,4)
%figure( 'Name', '类型二销售单价与销售量' );
plot( fitresult, xData, yData, excludedPoints );
legend('实际值', '离群值', '拟合值', 'Location', 'NorthEast', 'Interpreter', 'none' );
% 为坐标区加标签
xlabel( '', 'Interpreter', 'none' );
ylabel( '', 'Interpreter', 'none' );
title('类型二销售单价与预期销售量','FontSize',6)
xlim([0,20])
grid on
```

JB 检验相关代码以及 Spearman 相关系数代码。

```
clear,clc
load Fit_Data.mat

% 对数据进行JB正态分布检验
for i=1:3
[H(i),P(i)] = jbtest(Fit_Data(:,i),0.05);

end

% 输出正态分布检验结果，均为1，拒绝正态分布假设
disp(H)
disp(P)

%故选择Spearman相关系数
corr(Fit_Data,'type','Spearman')
```

4.6 互补性与替代性优化

互补性与替代性优化 Lingo 代码。

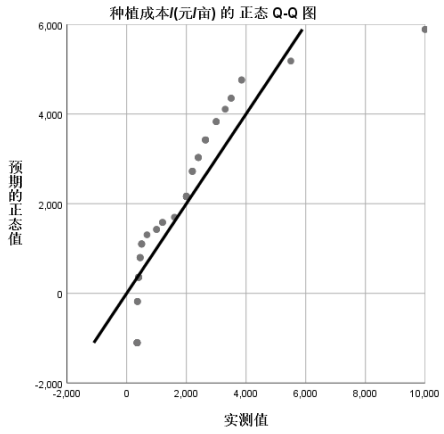
```
sets:
type/1..5/:AreaI,AreaII,PPrdt,PrdtI,PrdtII,Sold,Cost,WillSold;
link(type,type):weight;
endsets

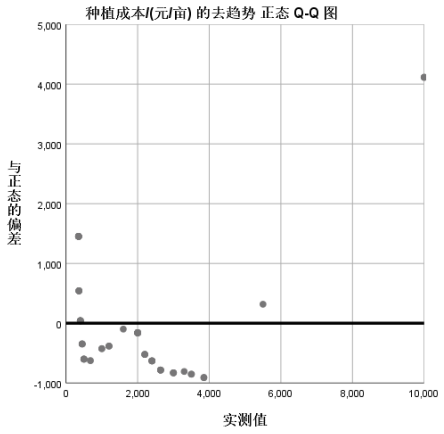
data:
weight=
0 -0.3 0.5 0.3 0.6
-0.3 0 0.4 0.2 0.4
0.5 0.4 0 -0.1 -0.2
0.3 0.2 -0.1 0 0.4
0.6 0.4 -0.2 0.4 0;
AreaI=
20 20 20 20 20;
PPrdt=
800 400 2700 300 10000;
Sold=
3.5 3.25 6.2 8 16;
Cost=
0.56 1 0.85 0.67 1;
enddata

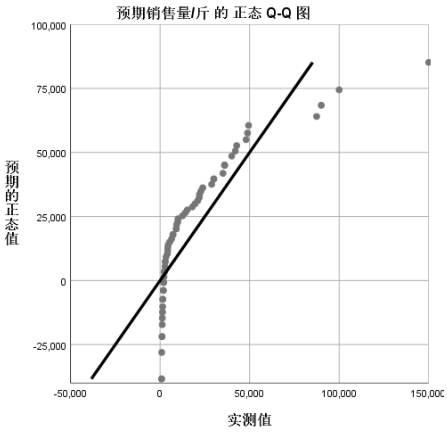
max=@sum(type(i):PrdtII(i)*(Sold(i)-Cost(i)));

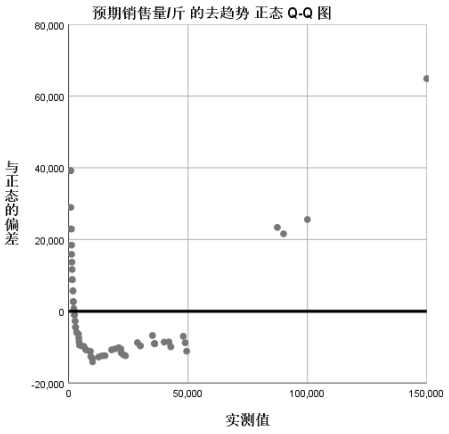
@for(type(i):WillSold=-4500*@log(Sold(i))+53000);
@for(type(i):PrdtII(i)<=WillSold(i));
@for(type(i):PrdtII(i)<=(@sum(type(j):PPrdt(j)*weight(i,j)*AreaII(j)*0.5)+AreaI(i)*PPrdt(i))
    );
@sum(type(i):AreaII(i))=100;
```

4.7 第三问 QQ 图展示


种植成本/(元/亩) 的 正态 Q-Q 图
预期的正态值
实测值
6,000
4,000
2,000
0
-2,000
-2,000
0
2,000
4,000
6,000
8,000
10,000


种植成本/(元/亩) 的去趋势 正态 Q-Q 图
与正态的偏差
实测值
5,000
4,000
3,000
2,000
1,000
0
-1,000
0
2,000
4,000
6,000
8,000
10,000


预期销售量/斤 的 正态 Q-Q 图
预期的正态值
实测值
100,000
75,000
50,000
25,000
0
-25,000
-50,000
0
50,000
100,000
150,000


预期销售量/斤 的去趋势 正态 Q-Q 图
与正态的偏差
实测值
80,000
60,000
40,000
20,000
0
-20,000
0
50,000
100,000
150,000